DESARROLLO WEB EN ENTORNO SERVIDOR

UT1. PROYECTO DE APLICACIÓN DE GENERACIÓN DE INFRAESTRUCTURAS CON IA

PARTE B: DOCUMENTACIÓN DEL PROYECTO ANÁLISIS FUNCIONAL

Fase 1: Preparación del entorno y análisis funcional

Nombre del proyecto	***Cloud 9 Systems***
Integrantes	Yoel José Bautista Díaz, Ángel León Santana, Juan Francisco Millet Acedo, Damián Pérez Alemán
Módulo	Desarrollo Web en Entorno Servidor
Curso	2026/2027
Centro	CIFP VILLA DE AGÜIMES
Fecha de entrega	14 / 10 / 2026
Profesor	YERAY ANDRÉS PÉREZ VALIENTE



Engineering your cloud.

→ÍNDICE←

1.1 Normas internas de trabajo....4
1.2 Control de versiones del documento....4
2. Descripción del problema y objetivos del sistema....5
2.1 Problema que se pretende resolver....5
2.2 Valor aportado por la aplicación....5
2.3 Objetivo principal....5
2.4 Objetivos secundarios....6
2.5 Alcance inicial y fuera de alcance....6
3. Tecnologías utilizadas y justificación....7
3.1 Dependencias y ficheros de configuración relevantes....8
4. Arquitectura MVC y ciclo de una petición Laravel....8
4.1 Arquitectura MVC....8
4.2 Ciclo de una petición GET / en Laravel....10
5. Organización de Git y GitHub....11
5.1 Estrategia de ramas....12
5.2 Política de commits y Pull Requests....12
5.3 Seguridad del repositorio....13
5.4 Evidencia de participación....14
6. Actores del sistema....15
6.1 Relaciones entre actores....16
7. Mapa funcional y módulos de la aplicación....17
7.1 Diagrama del mapa funcional....21
8. Requisitos y reglas de negocio....22
8.1 Requisitos funcionales (RF)....22
8.2 Requisitos no funcionales (RNF)....23
8.3 Reglas de negocio (RN)....24
8.4 Ambigüedades y decisiones pendientes....25
9. Análisis del dominio Cloud....26
9.1 Catálogo de productos y servicios....26
9.2 Proveedores y regiones....27
9.3 Proyectos y requerimientos estructurados....27
9.4 Carrito, presupuesto, pedido y pago simulado....28
9.5 Despliegues simulados y auditoría....29
10. Casos de uso....30
10.1 CU01 — Registro e inicio de sesión de usuario....31
10.2 CU02 — Consultar catálogo y filtrar recursos Cloud....32
10.3 CU03 — Gestionar recursos en el carrito de compra....33
10.4 CU04 — Crear y configurar un nuevo proyecto Cloud....34
10.5 CU05 — Introducir requerimientos en lenguaje natural (Procesamiento IA)....35

10.6 CU06 — Diseñar topología y aprobar arquitectura técnica....36
10.7 CU07 — Generar y congelar presupuesto formal....37
10.8 CU08 — Simular pasarela de pago y formalizar pedido....38
10.9 CU09 — Iniciar simulación de despliegue de infraestructura....39
10.10 CU10 — Monitorizar métricas y alertas del sistema....40
10.11 CU11 — Administrar y mantener el catálogo de recursos Cloud....41
10.12 CU12 — Consultar histórico y auditoría del proyecto....42
11. Modelo conceptual de datos....43
11.1 Entidades identificadas....43
11.2 Relaciones principales....45
11.3 Diagrama conceptual....46
11.4 Decisiones abiertas del modelo....46
12. Arquitectura Laravel inicial....47
12.1 Separación de responsabilidades....49
12.2 Diagrama de arquitectura inicial....50
13. Análisis del uso futuro de Inteligencia Artificial....51
13.1 Funciones previstas de la IA....51
13.2 Entradas y salidas previstas....51
13.3 Límites de la IA....51
13.4 Riesgos de la integración....52
14. Riesgos técnicos, seguridad y privacidad....53
14.1 Seguridad mínima del repositorio....55
15. Planificación inicial....56
15.1 Backlog inicial de grandes tareas....56
16. Conclusiones....57
16.1 Estado del proyecto al finalizar UT1....57
16.2 Principales decisiones tomadas....57
16.3 Aspectos que deberán revisarse en UT2....58
16.4 Reflexión del equipo....58
17. Referencias y fuentes consultadas....59
18. Registro de decisiones del proyecto....60

1.1 Normas internas de trabajo

- **Reparto de tareas y coordinación:** Las tareas se gestionan mediante el repositorio de GitHub utilizando Issues y Projects para dar seguimiento a cada entrega. Cada Integrante es responsable principal de su área (Documentación, Backend, Analista de Dominio y Frontend), coordinando las decisiones de diseño general de forma conjunta.

- **Canal de comunicación y seguimiento**: La comunicación diaria se realiza a través de un canal de Whatsapp para resolver dudas técnicas de forma ágil, complementado con reuniones de seguimiento para revisar el estado del código y de la documentación antes de cada fusión.

- **Resolución de conflictos y desviaciones:** Antes desacuerdos técnicos o metodológicos, prima el consenso basado en las especificaciones del proyecto. Si persiste la discrepancia, se realiza una votación interna. Ante el incumplimiento no justificado de plazos,se reestructuran las cargas de trabajo para no comprometer las entregas oficiales.

1.2 Control de versiones del documento

Versión	Fecha	Autor/es	Cambios realizados
0.1	05/10/2026	Responsable de Documentación	Borrador inicial de la estructura, requisitos, reglas de negocio y actores.
0.2	10/10/2026	Todo el equipo	Integración de la tabla de integrantes, mapa funcional, catálogo y modelo conceptual.
0.3	12/10/2026	Todo el equipo	Completar apartados y documentación y formatear.
0.4	13/10/2026	Backend / Entorno	Terminar tareas de entorno y backend
1.0	13/10/2026	Todo el equipo	Versión final revisada de la UT1 lista para la entrega oficial en formato PDF.

2. Descripción del problema y objetivos del sistema

2.1 Problema que se pretende resolver

Diseñar y presupuestar una infraestructura Cloud para desplegar un proyecto de software resulta altamente complejo para usuarios no expertos, desarrolladores independientes y PYMEs.

- **Falta de abstracción técnica:** Desconocimiento de la cantidad de recursos (CPU, memoria RAM, motor de base de datos o almacenamiento) necesarios para soportar una carga de trabajo determinada (por ejemplo, una aplicación web en Laravel para 500 usuarios concurrentes con soberanía de datos en la UE).

- **Imprevisibilidad de costes:** Dificultad para estimar costes recurrentes mensuales frente a costes únicos de instalación, lo que suele derivar en sorpresas presupuestarias.

- **Complejidad en reglas de compatibilidad:** Riesgo de diseñar arquitecturas incoherentes o inseguras por omitir componentes esenciales como grupos de seguridad, firewalls, redes privadas o sistemas de copias de seguridad.

2.2 Valor aportado por la aplicación

A diferencia de una tienda electrónica convencional que vende productos aislados o de una simple calculadora estática de precios, esta plataforma ofrece un entorno interactivo y guiado para el diseño y simulación de infraestructuras:

- Traducción inteligente de requisitos: Convierte las necesidades expresadas en lenguaje natural o parámetros sencillos en una propuesta estructurada de servidores, bases de datos, redes y backups.

- Determinismo y control estricto de costes: Aunque utiliza inteligencia artificial como motor de recomendación, la lógica de negocio de Laravel actúa como la única autoridad determinista encargada de validar compatibilidades, aplicar tarifas oficiales y activar alertas automáticas (como el control del presupuesto máximo de 50 €/mes).

- Simulación completa del ciclo de vida: Permite llevar una arquitectura desde la fase de borrador hasta la generación de un presupuesto formal inmutable, la simulación de pago y la visualización de un panel de recursos desplegados sin incurrir en costes reales de infraestructura.

2.3 Objetivo principal

Desarrollar una plataforma web basada en el framework Laravel que permita a los usuarios diseñar, configurar, presupuestar y simular el despliegue de infraestructuras Cloud personalizadas a partir de sus requisitos tecnológicos y restricciones económicas, combinando un asistente de recomendación basado en IA con un motor determinista de validación y cálculo de precios.

2.4 Objetivos secundarios

- **Catálogo unificado y estructurado:** Diseñar e implementar un catálogo de recursos Cloud (computación, bases de datos, almacenamiento, redes, seguridad e IA) categorizado por proveedores y regiones geográficas.

- **Motor de cálculo y presupuestación:** Construir un módulo de cálculo de presupuestos con control de alertas por exceso de coste (límite por defecto de 50 €/mes) y mecanismo de congelación temporal de precios durante 15 días.

- **Control de acceso y auditoría:** Implementar un sistema de roles y permisos (Cliente, Arquitecto, Auditor y Administrador) con registro automatizado de acciones críticas para garantizar la trazabilidad.

- **Módulo de recomendaciones por IA:** Integrar una API de IA restringida por validaciones backend estrictas que evite alucinaciones o sugerencias de productos inexistentes.

- **Simulación de gestión y despliegue:** Proporcionar un cuadro de mando donde el usuario pueda consultar sus proyectos activos, presupuestos emitidos y el estado simulado de sus recursos contratados

2.5 Alcance inicial y fuera de alcance

Incluido en el alcance	Fuera de alcance / fases posteriores
Registro de proyectos con especificación de usuarios, región y presupuesto.	Aprovisionamiento real de recursos en las APIs públicas de AWS, Azure o GCP.
Catálogo de productos simulados con filtrado por tipo, proveedor y región.	Procesamiento de cobros reales mediante pasarelas de pago bancarias.
Recomendación asistida por IA validada por lógica de negocio en Laravel.	Generación e integración de scripts automáticos de Terraform o Ansible.
Generación de presupuestos inmutables con IVA y desglose de costes.	Monitorización de métricas de rendimiento (CPU, RAM) en tiempo real sobre servidores.
Simulación de proceso de pago, generación de pedido y panel de despliegue.	Arquitectura multitenancy avanzada para el aislamiento de organizaciones.
Registro de auditoría de operaciones críticas y control de acceso basado en roles.	Migración automatizada de datos entre diferentes proveedores de nube.

3. Tecnologías utilizadas y justificación

- ***En esta tabla enumeramos las herramientas. Con cada tecnología explicamos qué función tiene dentro del proyecto y por qué se utiliza.***

Tecnología	Versión	Función en el proyecto	Justificación
Ubuntu	Ubuntu-24.04	Sistema Operativo, base de todo el entorno	Es el prototipo real de producción, además de que es gratuito estable y compatible con Laravel
Visual Studio Code	1.138.0	Editor con terminal y git integrado	Este editor permite ejecutar los servicios necesarios así como integrarse con git y poseer extensiones útiles
VirtualBox	7.2.18	Software donde virtualizamos nuestro entorno sin tocar el sistema anfitrión	Lo hemos elegido por que nos permite tener un entorno aislado y reproducible, proporciona seguridad y portabilidad
MariaDB	10.6.23	Sistema gestor de base de datos con codigo abierto que deriva de MySQL	Lo hemos elegido por que encaja con la naturaleza de nuestra aplicación, aporta integridad en las transacciones y es compatible con laravel
Git - GitHub	2.34.1	Herramientas de control de versiones	Permite un trabajo en equipo y un flujo definido por ramas, asi tenemos una trazabilidad de los cambios y evaluamos si los pasamos a producción o no
Laravel	10.50.3	Es el FrameWork de PHP que debemos usar para el proyecto	Es orientado al backend Define una estructura clara Facilita modelar y versionar entidades Incluye funcionalidades futuras

3.1 Dependencias y ficheros de configuración relevantes

Para este caso hemos separado dos tipos, los que se suben a GitHub y los que no se suben:

Se sube a GitHub	
Elemento	**Funcion**
composer.json	Declaramos dependencias de PHP y autoload
composer.lock	Fija las versiones de las dependencias
package.json	Declara dependencias y scripts del FrontEnd
.env.example	Plantilla de .env
.gitignore	Define lo que Git ignora
No se Sube a GitHub	
vendor	Código de las dependencias de PHP
.env	Configuración local, contraseñas, etc

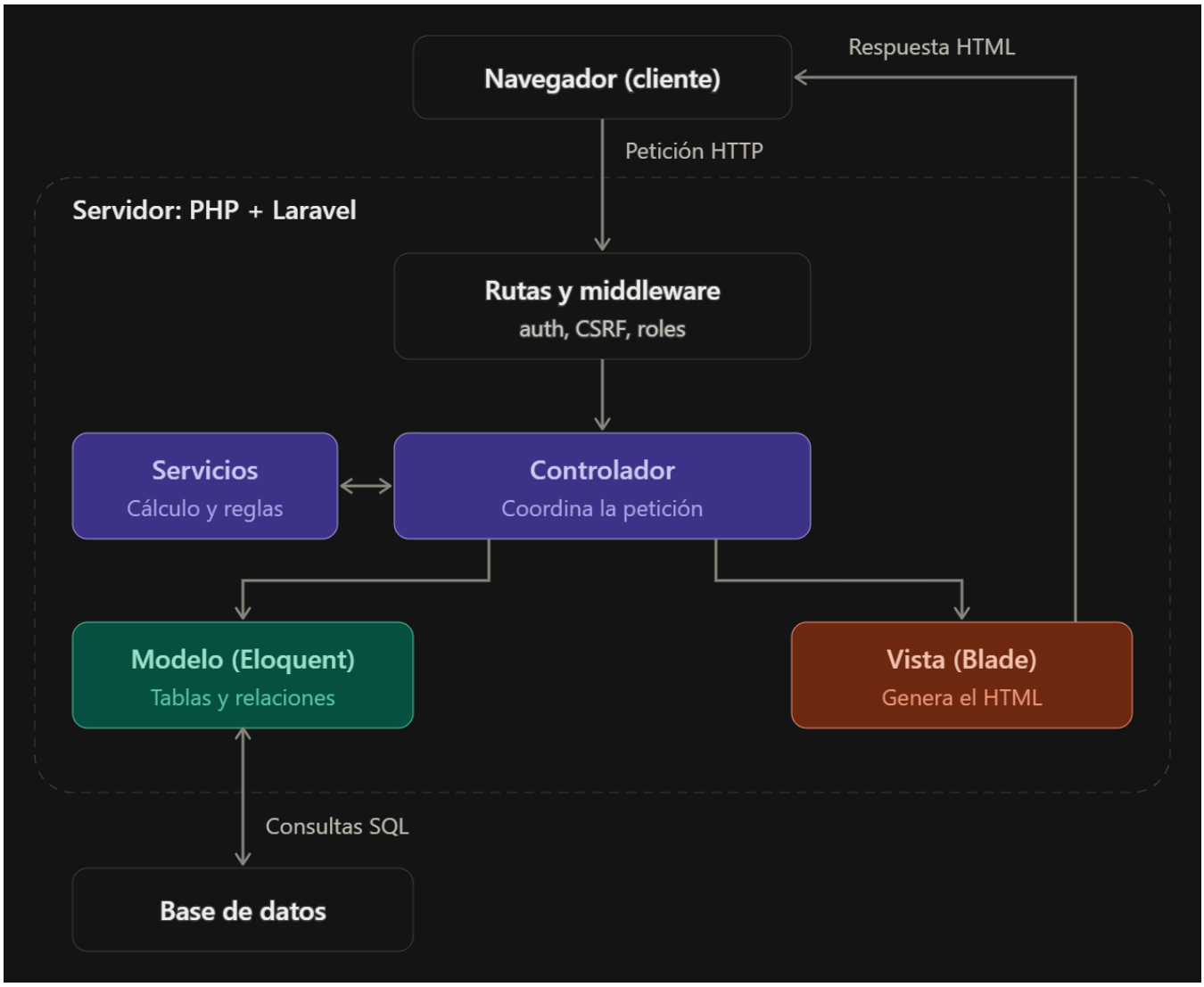
4. Arquitectura MVC y ciclo de una petición Laravel

4.1 Arquitectura MVC

Explicamos qué parte corresponde al cliente, qué parte se ejecuta en el servidor y qué papel tienen HTTP, PHP, Laravel y la base de datos.

Capa	Donde se Ejecuta	Responsabilidad
HTTP / HTTPS	Se ejecuta en ambos	Transportar peticiones y respuestas
PHP	Servidor	Ejecuta el código de la APP
Laravel	Servidor	Rutas, validaciones, lógica de negocio, seguridad, etc etc
Base de datos	Servidor o Servicio externo	Giardia y recupera los datos

→Diagrama arquitectura MVC con protocolo HTTP←



4.2 Ciclo de una petición GET / en Laravel

Podemos ver una petición desde el →navegador, petición HTTP, servidor de desarrollo, public/index.php, autoload de Composer, arranque de Laravel, sistema de rutas, routes/web.php, generación de respuesta, respuesta HTTP y →representación en el navegador.

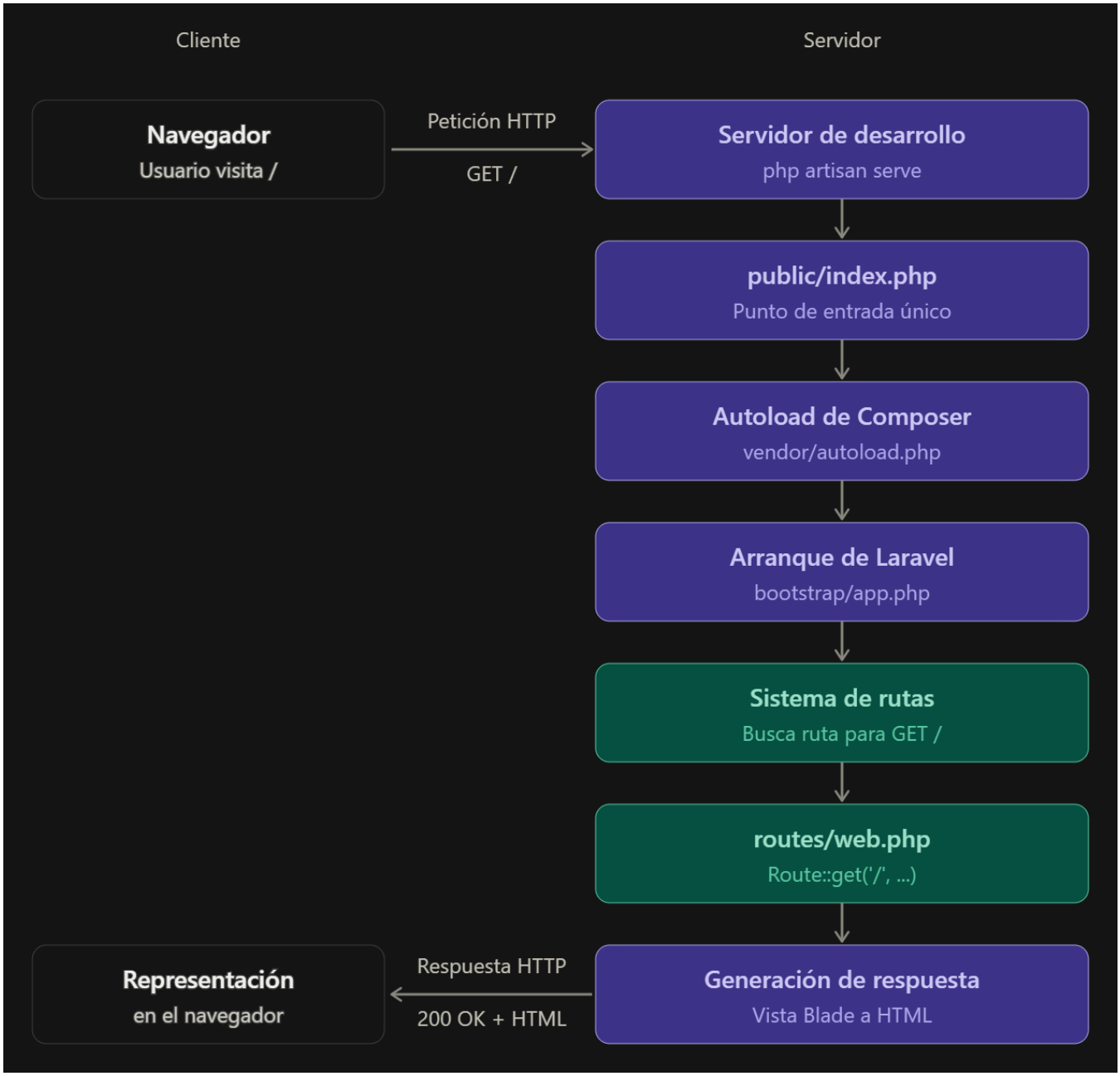
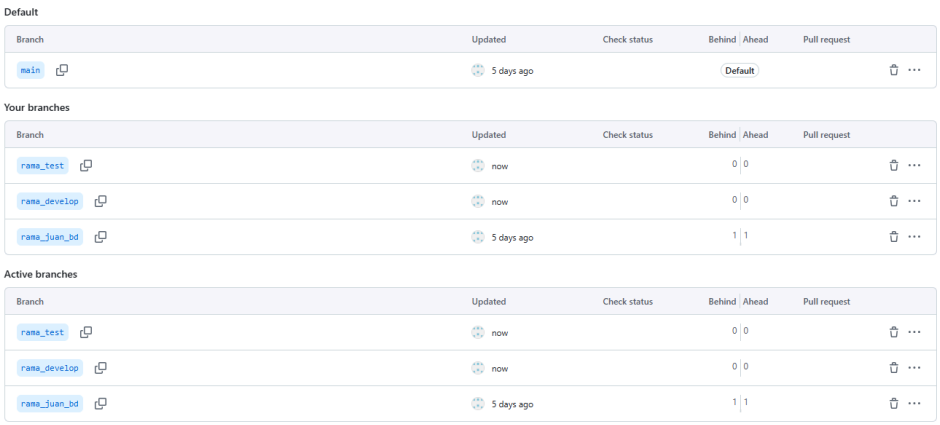


Tabla con el proceso de petición y respuesta:

Paso	Elemento	Explicación
1º	*Navegador*	Construye la petición HTTP con el método GET y la ruta
2º	*Petición HTTP*	Viaja por la red hará el servidor y le Indica que solo quiere consultar (GET) y no modificar
3º	*Servidor de desarrollo*	Recibe la petición y la entrega a PHP
4º	*public/index.php*	Es el punto de entrada de la aplicación
5º	*Autoload Composer*	Es el fichero que carga el index.php, esto permite que PHP encuentre las clases automáticamente
6º	*Arranque de Laravel*	Se carga en boostrap/app.php. esto crea la aplicación y el contenedor de servicios, básicamente se convierte en una Request o petición
7º	*Sistema de Rutas*	Compara el GET y las rutas registradas, si la encuentra y puedes acceder entras
8º	*routes/web.php*	Es el fichero donde se declaran las rutas
9º	*Generación de Respuesta*	La petición devuelve una vista blade que complica la plantilla HTML Laravel
10º	*Respuesta HTTP*	El servidor envía al navegador el código 200 (ok), las cabeceras y el cuerpo, terminando aquí el ciclo
11º	*Representación en el navegador*	El navegador interpreta el HTML, solicita los recursos adicionales con su propia petición get y devuelve la página en pantalla

5. Organización de Git y GitHub



5.1 Estrategia de ramas

Para el desarrollo del proyecto se utiliza una adaptación del modelo Git Flow para garantizar la estabilidad del código en producción. Las ramas del repositorio se organizan de la siguiente manera:

- **main:** Es la rama por defecto y principal. Contiene única y exclusivamente el código que ya está en producción, es totalmente estable y ha sido testeado.
- **rama_develop:** Es la rama de integración. Aquí se fusionan todas las nuevas funcionalidades terminadas para centralizar el desarrollo antes de pasar a la fase de pruebas.
- **rama_test:** Dedicada al entorno de pruebas (Staging). Sirve para desplegar el código de integración y realizar pruebas funcionales completas (QA) sin comprometer la producción.
- **Ramas de característica** *(feature/... o individuales*): Ramas temporales creadas para el desarrollo de tareas específicas, módulos o asignaciones por desarrollador (por ejemplo, **rama_juan_bd**). Se crean a partir d**e rama_develop** y, una vez finalizada la tarea, se reintegran en ella mediante una Pull Request.

5.2 Política de commits y Pull Requests

Para mantener la consistencia y la calidad del código en nuestro proyecto Laravel, se establece la siguiente política de contribución:

Convención de Commits (Conventional Commits)

Los mensajes de los commits deben seguir una estructura clara que identifique el propósito del cambio. Usaremos el formato: tipo(componente): descripción corta en minúsculas.

- **feat:** Nueva funcionalidad para el proyecto (ej. feat(auth): implementar login con dos factores).
- **fix:** Corrección de un error o bug (ej. fix(user): corregir validación del correo electrónico en el registro).
- **docs**: Cambios solo en la documentación (ej. docs(readme): actualizar instrucciones de instalación).
- **style**: Cambios que no afectan el significado del código (espacios en blanco, formato, punto y coma faltante, etc.).
- **refactor**: Un cambio de código que no corrige un error ni añade una funcionalidad (ej. optimizar una consulta de Eloquent).
- **test:** Añadir pruebas faltantes o corregir pruebas existentes (ej. test(feature): agregar test para el controlador de productos).

Revisión de Pull Requests (PR)

- Quién revisa: Todo Pull Request dirigido a rama_develop, rama_test o main debe ser revisado y aprobado por al menos un Desarrollador Senior o el Tech Lead del proyecto.
- Proceso: El creador del PR no puede fusionar su propio código. Se requiere la validación de la integración continua (si aplica) y la aprobación visual del revisor para asegurar que se sigan las buenas prácticas de Laravel (Uso de Request Classes, Service Pattern, etc.).

Resolución de Conflictos

Cuando existan conflictos de código al intentar fusionar una rama, se procederá de la siguiente manera:

1. El desarrollador responsable de la característica deberá traer los cambios más recientes de la rama de destino a su rama local utilizando git fetch y git merge (o git rebase según se acuerde en el equipo).
2. La resolución de conflictos se hará localmente en el IDE, asegurando no borrar lógica importante de otros compañeros.
3. Si existen dudas sobre qué código debe prevalecer, se debe consultar directamente con el autor de las líneas en conflicto antes de subir los cambios actualizados

5.3 Seguridad del repositorio

Para garantizar la integridad del código y proteger la información sensible del proyecto, se aplican de forma obligatoria las siguientes directrices de seguridad:

- ***Exclusión del archivo .env****:* El archivo de configuración de entorno .env nunca se sube al repositorio bajo ninguna circunstancia. Este archivo contiene las credenciales locales de la base de datos, claves de cifrado de Laravel (APP_KEY) y accesos a servicios de terceros que deben permanecer estrictamente confidenciales.
- ***Uso correcto de .env.example***: El archivo .env.example sirve únicamente como plantilla para que otros desarrolladores configuren su entorno local. No debe contener secretos, contraseñas reales ni claves de producción. Solo incluirá las variables vacías o con valores de demostración por defecto.
- ***Auditoría del archivo .gitignore***: Se debe revisar periódicamente el archivo .gitignore para asegurar que las carpetas del sistema, dependencias de Composer (/vendor), paquetes de Node (/node_modules) y archivos temporales o de almacenamiento (/storage/*.log, /public/storage) estén correctamente configurados para no ser rastreados por Git.
- ***Prohibición estricta de credenciales en el código:*** No se almacenan contraseñas, tokens, API Keys, claves privadas ni credenciales directamente en el código fuente (hardcoded). Cualquier clave de acceso requerida por la aplicación debe ser inyectada dinámicamente a través del archivo de configuración correspondiente en la carpeta config/, leyendo siempre su valor desde el archivo .env mediante la función auxiliar env().

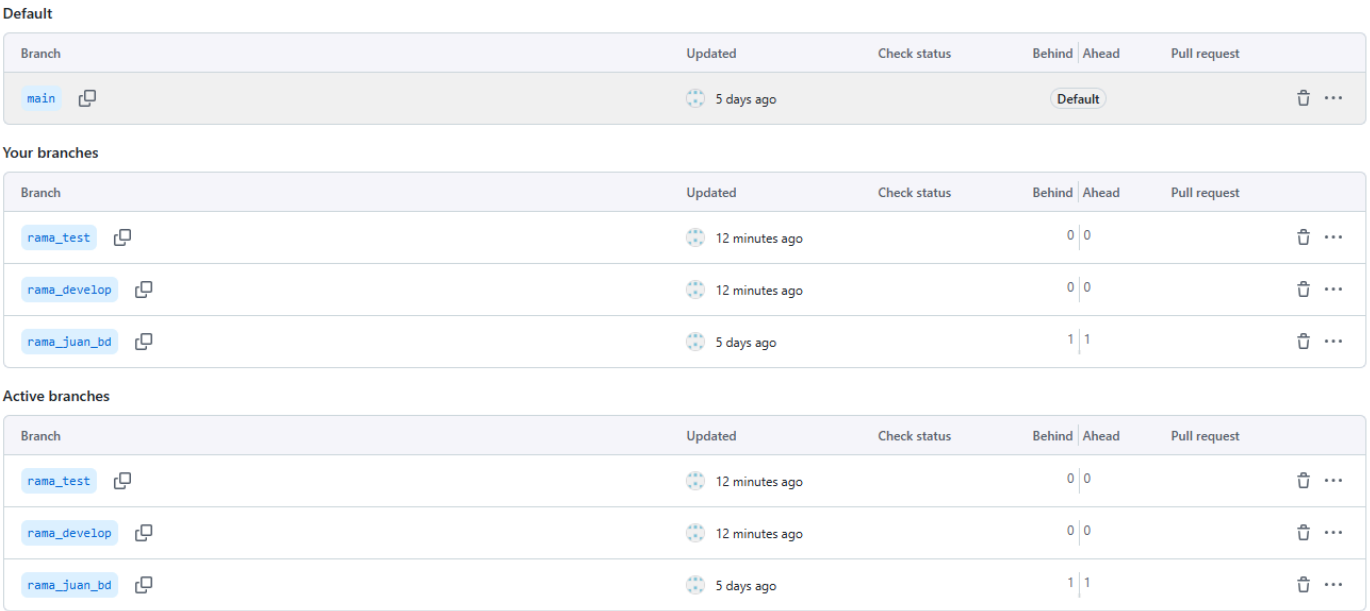
5.4 Evidencia de participación

Actualmente, el proyecto se encuentra en su fase inicial de planificación, arquitectura y diseño conceptual, por lo que aún no se ha dado inicio a la etapa de programación o desarrollo de código fuente.

La participación de los integrantes del equipo durante este periodo se ve reflejada en los siguientes entregables base depositados en el repositorio:

- **Modelado de Base de Datos:** Creación, normalización y estructuración de los diagramas entidad-relación y los scripts iniciales para la base de datos.
- **Diagramas de Clases:** Definición de la estructura lógica, propiedades y métodos iniciales que guiarán el posterior desarrollo en Laravel.
- **Estructura Documental:** Elaboración y actualización constante de la documentación técnica del proyecto.

A medida que se avance hacia la fase de codificación, **esta sección se irá completando de forma progresiva** mediante la inclusión de capturas de pantalla del historial de commits, registros de contribución en la plataforma de control de versiones y el tablero de asignación de tareas del equipo.



Default

Branch	Updated	Check status	Behind	Ahead	Pull request
main	5 days ago		Default		

Your branches

Branch	Updated	Check status	Behind	Ahead	Pull request
rama_test	12 minutes ago		0	0	
rama_develop	12 minutes ago		0	0	
rama_juan_bd	5 days ago		1	1	

Active branches

Branch	Updated	Check status	Behind	Ahead	Pull request
rama_test	12 minutes ago		0	0	
rama_develop	12 minutes ago		0	0	
rama_juan_bd	5 days ago		1	1	

6. Actores del sistema

- ***A continuación analizamos los cuatro perfiles iniciales :***

Actor	Objetivo	Funcionalidades principales	Información consultada	Información modificada
CLIENTE	Solicitar servicios, hacer seguimiento de sus proyectos y realizar pagos.	Crear solicitudes de presupuesto, ver el estado de sus proyectos.	Catálogo de servicios, estado de sus proyectos encargados, presupuestos asignados.	Planificaciones, entregables del proyecto, diagramas técnicos y estado de avance de la obra/código.
ARQUITECTO	Diseñar, planificar y gestionar el desarrollo técnico de los proyectos asignados.	Asignar tareas, subir planos o diagramas de arquitectura, y reportar avances del proyecto	Requisitos del cliente, asignación de recursos y estado de las tareas técnicas.	Planificaciones, entregables del proyecto, diagramas técnicos y estado de avance de la obra/código.
AUDITOR	Supervisar que los procesos del sistema, la seguridad y los proyectos cumplan con las normativas y estándares exigidos	Revisar historiales de acciones, generar reportes de cumplimiento y validar la calidad del trabajo.	Logs de actividad del sistema, registros de cambios, datos financieros y reportes de calidad	Estado de las revisiones (Aprobado/Rechazado) y notas o informes de auditoría.
ADMINISTRADOR	Gestionar el correcto funcionamiento global de la plataforma, los usuarios y la seguridad del sistema.	Dar de alta/baja a usuarios, gestionar los permisos de los roles, y mantener las tablas maestras de la base de datos.	Lista completa de usuarios, métricas de rendimiento del sistema y alertas de errores.	Permisos de usuario, configuraciones globales del sistema y estados de cuentas bloqueadas.s

6.1 Relaciones entre actores

1. Responsabilidades compartidas (Colaboración)

- **Cliente ↔ Arquitecto:** Comparten el seguimiento del ciclo de vida del proyecto. El **Cliente** define los requisitos iniciales y el **Arquitecto** los traduce en especificaciones técnicas. Ambos interactúan sobre el mismo flujo de estados del proyecto (ej. *En Revisión, Aprobado, En Desarrollo*).
- **Administrador ↔ Auditor:** Ambos velan por la seguridad del sistema, pero desde perspectivas distintas. Comparten el acceso a las métricas del sistema y de uso, aunque el Administrador lo hace de forma operativa y el Auditor de forma analítica.

2. Operaciones exclusivas (Privilegios únicos)

- **Administrador:** Es el único actor con permisos para **gestionar usuarios** (dar de alta, baja o modificar roles) y alterar la configuración global de la plataforma. Ningún otro rol puede acceder a estas funciones.
- **Auditor:** Posee acceso exclusivo a los **informes de cumplimiento y logs de cambios históricos**. Su entorno es estrictamente de *solo lectura* sobre los datos del proyecto, garantizando que su evaluación sea neutral.
- **Cliente:** Es el único que puede **iniciar pasarelas de pago** y emitir valoraciones de satisfacción sobre el servicio recibido.

7. Mapa funcional y módulos de la aplicación

- ***Identificamos los grandes módulos sin diseñar todavía todos los detalles:***.

1. Módulo: Usuarios

- **Objetivo:** Gestionar las cuentas de acceso, el registro y el control de roles (RBAC) en la plataforma.
- **Actores:** Todos (Cliente, Arquitecto, Auditor, Administrador).
- **Datos principales:** ID, nombre, email, contraseña cifrada, rol, estado de cuenta.
- **Operaciones:** Registrar, iniciar sesión, modificar perfil, dar de baja, asignar roles.
- **Relaciones:** Pedidos y Carrito.
- **Riesgos / reglas:** Suplantación de identidad o robo de cuentas. *Regla:* Es obligatorio el cifrado de contraseñas en el servidor y un usuario solo puede tener un rol activo a la vez (RBAC).

2. Módulo: Catálogo

- **Objetivo:** Mostrar de forma estructurada los servicios Cloud simulados y recursos disponibles.
- **Actores:** Cliente, Arquitecto, Administrador.
- **Datos principales:** ID_Servicio, nombre, descripción técnica, precio base, región.
- **Operaciones:** Crear servicio, listar catálogo, actualizar tarifas, filtrar recursos.
- **Relaciones:** Recursos.
- **Riesgos / reglas:** Inconsistencia de tarifas. *Regla:* Las actualizaciones de precios en los recursos del catálogo no deben afectar a los carritos que los clientes ya tengan abiertos en ese momento.

3. Módulo: Proveedores / regiones

- **Objetivo:** Administrar las zonas geográficas simuladas y los proveedores de infraestructura ficticios (AWS, Azure, etc.).
- **Actores:** Administrador, Arquitecto.
- **Datos principales:** ID_Region, proveedor, latencia simulada, multiplicador de coste.
- **Operaciones:** Añadir región, deshabilitar proveedor, consultar costes por zona.
- **Relaciones:** Recursos.
- **Riesgos / reglas:** Caídas de servicio simuladas o latencias altas. *Regla:* Si una región se marca como "Inactiva", el sistema debe pausar automáticamente la contratación de recursos asociados a ella.

4. Módulo: Proyectos

- **Objetivo:** Organizar los entornos de trabajo o despliegues solicitados por los clientes.
- **Actores:** Cliente, Arquitecto, Administrador.
- **Datos principales:** ID_Proyecto, nombre, estado, cliente_asociado, fecha_creación.
- **Operaciones:** Crear proyecto, asignar arquitecto, cambiar estado, archivar proyecto.
- **Relaciones:** Usuarios y Requerimientos.
- **Riesgos / reglas:** Registros huérfanos en la base de datos. *Regla:* No se puede eliminar a un usuario del sistema si tiene proyectos activos asociados sin antes haberlos archivado o transferido.

5. Módulo: Requerimientos

- **Objetivo:** Documentar y validar las necesidades del cliente previas al diseño de la infraestructura.
- **Actores:** Cliente, Arquitecto.
- **Datos principales:** ID_Req, descripción, prioridad, estado validación.
- **Operaciones:** Subir requerimiento, validar, rechazar, añadir comentarios.
- **Relaciones:** Proyectos y Arquitecturas.
- **Riesgos / reglas:** Ambigüedad en la definición técnica. *Regla:* El flujo del proyecto quedará bloqueado y no podrá avanzar a la fase de diseño si existe algún requerimiento en estado "Rechazado".

6. Módulo: Arquitecturas

- **Objetivo:** Diseñar técnicamente la topología y planos lógicos de la infraestructura Cloud solicitada.
- **Actores:** Arquitecto, Cliente.
- **Datos principales:** ID_Arq, diagrama, especificaciones_hardware, ID_Proyecto.
- **Operaciones:** Diseñar planos, asociar recursos, exportar topología, aprobar diseño.
- **Relaciones:** Requerimientos y Catálogo.
- **Riesgos / reglas:** Incompatibilidad de componentes Cloud diseñados. *Regla:* Una arquitectura solo puede pasar a la fase de presupuestación si cuenta con el estado de "Aprobada por el Cliente".

7. Módulo: Carrito

- **Objetivo:** Almacenar temporalmente la selección de recursos Cloud antes de consolidar la compra.
- **Actores:** Cliente.
- **Datos principales:** ID_Carrito, lista_items, coste_estimado_total.
- **Operaciones:** Añadir recurso, eliminar ítem, vaciar carrito, calcular total.
- **Relaciones:** Usuarios y Catálogo.
- **Riesgos / reglas:** Persistencia infinita que sature el servidor. *Regla:* El carrito está ligado a la sesión del usuario; si hay inactividad prolongada (ej. 7 días), el contenido se vaciará automáticamente.

8. Módulo: Presupuestos

- **Objetivo:** Congelar las tarifas comerciales de la arquitectura diseñada durante un plazo garantizado.
- **Actores:** Cliente, Arquitecto, Administrador.
- **Datos principales:** ID_Presupuesto, desglose_precios, fecha_emisión, fecha_caducidad.
- **Operaciones:** Generar presupuesto, congelar precios, enviar al cliente, rechazar.
- **Relaciones:** Proyectos, Usuarios y Carrito.
- **Riesgos / reglas:** Pérdida de margen comercial por fluctuaciones del mercado. *Regla:* Inmutabilidad absoluta de los precios congelados desde el momento de su emisión hasta su fecha de caducidad.

9. Módulo: Pedidos

- **Objetivo:** Formalizar la intención de compra del cliente en base a un presupuesto aprobado.
- **Actores:** Cliente, Administrador.
- **Datos principales:** ID_Pedido, ID_Presupuesto, fecha_pedido, estado (Pendiente/Procesado).
- **Operaciones:** Crear pedido, cancelar orden, verificar estado.
- **Relaciones:** Usuarios y Presupuestos.
- **Riesgos / reglas:** Generación de órdenes duplicadas por fallos de red. *Regla:* Requiere cumplimiento estricto de atomicidad (ACID); no puede crearse un pedido si el presupuesto asociado ya ha expirado.

10. Módulo: Pagos

- **Objetivo:** Procesar y simular las transacciones financieras del proyecto. *(Nota: Corregido el objetivo según el contexto correspondiente).*
- **Actores:** Arquitecto, Administrador.
- **Datos principales:** ID_Pago, ID_Pedido, monto, estado_transaccion.
- **Operaciones:** Simular pasarela de pago, emitir factura ficticia, verificar fondos.
- **Relaciones:** Pedidos.
- **Riesgos / reglas:** Desincronización entre la pasarela y la base de datos (pago cobrado pero

pedido no activo). *Regla:* Si la transacción simulada falla, se debe aplicar un rollback inmediato del estado.

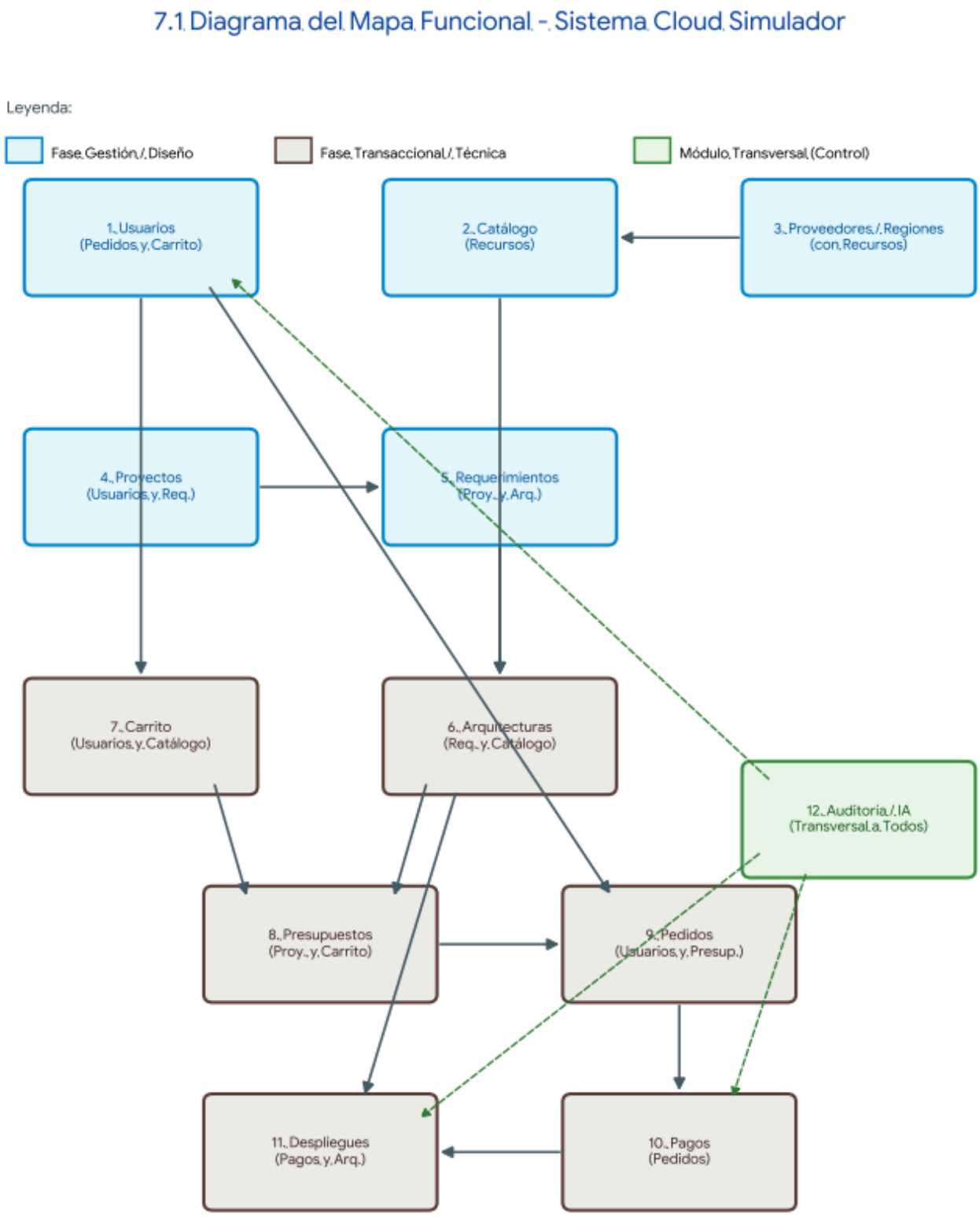
11. Módulo: Despliegues

- **Objetivo:** Ejecutar de forma simulada el aprovisionamiento de la infraestructura técnica contratada.
- **Actores:** Arquitecto, Administrador.
- **Datos principales:** ID_Despliegue, log_ejecución, porcentaje_progreso, resultado.
- **Operaciones:** Iniciar despliegue simulado, pausar, testear entorno, reportar error.
- **Relaciones:** Pagos y Arquitecturas.
- **Riesgos / reglas:** Bucles infinitos o cuelgues durante el aprovisionamiento asíncrono. *Regla:* Si ocurre un error crítico en la simulación técnica, el sistema ejecutará un protocolo de desmantelamiento automático.

12. Módulo: Auditoría / IA

- **Objetivo:** Monitorizar la seguridad, registrar logs inmutables y ofrecer analíticas inteligentes del sistema.
- **Actores:** Auditor, Administrador.
- **Datos principales:** ID_Log, fecha_hora, IP, acción_realizada, reporte_IA.
- **Operaciones:** Registrar eventos, consultar logs, generar informe de calidad, detectar anomalías.
- **Relaciones:** Transversal con todos los módulos del sistema.
- **Riesgos / reglas:** Manipulación fraudulenta del historial de acciones del sistema. *Regla:* La base de datos de logs debe ser de solo lectura e inserción (Append-Only). Nadie puede modificar ni borrar un registro.

7.1 Diagrama del mapa funcional



7.1 Diagrama funcional del sistema Cloud / S

8. Requisitos y reglas de negocio

8.1 Requisitos funcionales (RF)

ID	Requisito	Actor / origen	Prioridad	Criterio de aceptación
RF01	Permisión de navegación y selección de recursos cloud del catálogo sin autenticación previa.	Usuario Anónimo	Alta	El sistema permite añadir ítems al carrito a visitantes no logueados.
RF02	Registro e inicio de sesión de usuarios en la plataforma.	Usuario Anónimo / Sistema	Alta	El formulario valida el correo y la contraseña, creando un perfil en la base de datos.
RF03	Persistencia del carrito de compra para usuarios autenticados en base de datos.	Usuario Registrado	Alta	Si el usuario cierra sesión o cambia de dispositivo, los productos del carrito siguen guardados.
RF04	Volatilidad del carrito de compra para sesiones de usuarios no autenticados.	Usuario Anónimo / Sistema	Media	Al cerrar el navegador o expirar la sesión cookies/sessionStorage sin registrarse, el carrito se borra.
RF05	Flujo de registro obligatorio durante el proceso de confirmación de pedido.	Usuario Anónimo	Alta	Al pulsar "Confirmar pedido", el sistema exige registrarse o loguearse antes de proceder al pago.
RF06	Bloqueo de carritos múltiples simultáneos, garantizando un único carrito activo por usuario.	Sistema	Alta	El sistema impide por diseño que un identificador de usuario posea más de un carrito activo.
RF07	Detección de conflicto de carritos duplicados al iniciar sesión (Login Post-Carrito).	Sistema	Alta	Si un usuario anónimo con ítems en el carrito se logea y ya tenía un carrito guardado, el sistema frena el flujo y lanza un aviso.

RF08	Resolución de conflicto de carritos: Opción de conservar el carrito histórico.	Usuario Registrado	Alta	Al elegir "Continuar con mi carrito anterior", el sistema recupera los datos antiguos y borra los del carrito anónimo.
RF09	Resolución de conflicto de carritos: Opción de sobrescribir con el carrito nuevo.	Usuario Registrado	Alta	Al elegir "Continuar con el nuevo carrito", el sistema asocia los ítems anónimos al usuario y descarta/archiva el anterior.
RF10	Conversión automatizada de Carrito a Pedido tras la validación de pago.	Sistema / Pasarela	Alta	Al procesar el pago correctamente, el carrito pasa a estado "Pedido", se congela su edición y se vacía el carrito activo

8.2 Requisitos no funcionales (RNF)

ID	Requisito	Categoría	Cómo se comprobará
RNF01	La aplicación e-commerce debe garantizar una **disponibilidad mínima del 99.9%** (SLA) de forma anual.	Disponibilidad / Fiabilidad	Mediante herramientas de monitorización cloud (como AWS CloudWatch o Grafana) comprobando el porcentaje de uptime del servidor de producción.
RNF02	El procesamiento de la IA para leer el textarea y generar el carrito debe responder en **menos de 3 segundos** bajo condiciones normales.	Rendimiento / Eficiencia	Realizando pruebas de carga automatizadas con herramientas de testing de rendimiento (como Apache JMeter o Locust).
RNF03	Todas las comunicaciones y credenciales de usuarios deben transmitirse de forma cifrada mediante **HTTPS (TLS 1.3)** y contraseñas hasheadas con **bcrypt**.	Seguridad	Mediante una auditoría de seguridad externa o pasándole un escáner de vulnerabilidades SSL automatizado como Qualys SSL Labs.
RNF04	El sistema debe ser capaz de **escalar**	Escalabilidad	Simulando un pico masivo de tráfico

	horizontalmente de forma automática si el uso de CPU de los servidores supera el 70%.		concurrente y verificando en la consola cloud que se despliegan nuevas instancias automáticamente.
RNF05	Las copias de seguridad de las bases de datos de usuarios y pedidos deben realizarse de forma automatizada **cada 24 horas** con retención de 30 días.	Mantenibilidad / Respaldo	Verificando diariamente el bucket de almacenamiento cloud donde se guardan los archivos .bak o capturas de snapshot programadas

8.3 Reglas de negocio (RN)

D	Regla	Elementos afectados	Observaciones / casos límite
RN01	Un usuario no autenticado (anónimo) sólo puede mantener activo su carrito mientras dura su sesión de navegación.	CARRITO, USUARIOS	Al cerrar la pestaña del navegador o expirar la sesión por inactividad, el carrito se destruye automáticamente.
RN02	Si un usuario se logea teniendo un carrito anónimo y otro guardado históricamente, el sistema debe obligar a elegir cuál mantener mediante un aviso modal.	CARRITO, USUARIOS	Caso límite: Si el usuario cancela o cierra la ventana modal sin elegir, se mantendrá bloqueado el proceso de pago.
RN03	El TEXTAREA procesado por el SERVICIO_IA solo puede añadir al carrito recursos que existan explícitamente y estén activos en el CATALOGO.	SERVICIO_IA, CARRITO, CATALOGO	Si el cliente pide algo que no vendéis (ej: *"Quiero un café"*), la IA ignorará esa petición o devolverá un mensaje de error controlado.
RN04	Para poder asociar un FIREWALL o un servicio de MONITORIZACION, el carrito debe contener obligatoriamente al menos una red virtual (REDES) o un SERVIDOR contratado.	FIREWALL, MONITORIZACION, SERVIDOR, REDES	No se puede contratar seguridad perimetral o monitorización de forma aislada sin infraestructura base en el pedido.

RN05	Un pedido no puede pasar a estado "Aprobado" ni iniciar el despliegue si el pago no ha sido validado correctamente por la pasarela externa.	PEDIDOS, esquema_validar_pedido	Si la pasarela da error de saldo o tiempo de espera (Timeout), el pedido se queda en estado "Pendiente" y el carrito no se vacía.
RN06	Cada recurso cloud contratado en el carrito debe incluir de forma obligatoria parámetros de configuración mínimos (ej: vCPUs y RAM para SERVIDOR, o gigabytes para ALMACENAMIENTO).	CARRITO, SERVIDOR, ALMACENAMIENTO	Si la IA no logra deducir los parámetros mínimos del texto del usuario, el sistema asignará por defecto el plan más básico (Tier 1).
RN07	El sistema limita la contratación máxima por pedido a un tope de recursos por usuario para evitar abusos o costes descontrolados (ej: máximo 20 servidores o 50TB de almacenamiento).	CARRITO, PEDIDOS	Si el carrito supera el límite permitido, el botón de "Confirmar pedido" se deshabilita y se solicita contactar con SOPORTE

8.4 Ambigüedades y decisiones pendientes

Tema	Ambigüedad / duda	Decisión pendiente o acordada
Procesamiento del TEXTAREA	El texto libre del usuario puede ser confuso, contradictorio o incluir peticiones de recursos inexistentes.	**Acordada:** El SERVICIO_IA usará un prompt estricto para devolver exclusivamente un JSON estructurado. Si no entiende un recurso, lo ignorará y añadirá un campo de "advertencia" para avisar al cliente.
Precios de la IA	El coste de llamadas a APIs externas (OpenAI, Anthropic, etc.) por cada interacción con el TEXTAREA puede descontrolar el presupuesto.	**Pendiente:** Evaluar en la UT2 si se asume el coste mediante un modelo de suscripción premium para el cliente o si se despliega un modelo de código abierto local (tipo Llama) en nuestro propio servidor.
Configuración por defecto	Cuando el usuario pide un recurso de forma vaga (ej: *"Quiero un servidor"*), la IA no	**Acordada:** Ante la falta de detalles específicos en el texto, el sistema asignará automáticamente la configuración mínima del plan básico del CATÁLOGO.

	sabe cuánta RAM o vCPUs asignarle.	
Persistencia del carrito anónimo	No está cerrado al 100% cómo mantener el carrito de un usuario anónimo si este refresca la página por error.	**Acordada:** Se utilizará el almacenamiento local del navegador (localStorage o sessionStorage) para identificar temporalmente el carrito antes de que el usuario inicie sesión.
Validación de stock cloud	Al ser un e-commerce de servicios cloud virtuales, teóricamente no hay "stock" físico, pero nuestros propios servidores tienen un límite de capacidad.	**Pendiente:** Diseñar en el esquema_validar_pedido un sistema de cuotas máximas que verifique si nuestra propia infraestructura puede soportar los recursos que el cliente quiere levantar.
Pasarela de pago simulada	Cómo realizar las pruebas de cobro de los pedidos sin usar dinero real durante el desarrollo de la asignatura.	**Acordada:** Se integrará el entorno de pruebas (*Sandbox*) de una pasarela conocida (como Stripe o PayPal Developer) utilizando tarjetas de crédito ficticias para simular el éxito/error

9. Análisis del dominio Cloud

9.1 Catálogo de productos y servicios

Tipo	Información común	Información específica	Ejemplo de configuración
COMPUTACIÓN	Nombre, región, precio	CPU, RAM, almacenamiento, sistema operativo	VPS 2 CPU, 4 GB RAM, 80 GB SSD, Ubuntu
ALMACENAMIENTO	Nombre, proveedor, región, precio	Tipo, capacidad, rendimiento	SSD 100 GB
BASES DE DATOS	Nombre, proveedor, región, precio	Motor, versión, capacidad, conexiones	MySQL 8, 50 GB
SEGURIDAD	Nombre, proveedor, región, precio	Tipo de protección, reglas, tráfico protegido	Firewall con reglas de entrada/salida
BACKUP	Nombre, proveedor, región, precio	Frecuencia, retención, capacidad	Backup diario, retención 30 días
MONITORIZACIÓN	Nombre, proveedor,	Métricas, logs, alertas	CPU + RAM + disco +

	región, precio		alertas
SERVICIO _IA	Nombre, proveedor, región, precio	Modelo, capacidad de cómputo, memoria, uso	GPU para ejecutar modelo de IA
SOPORTE	Nombre, proveedor, precio	Nivel de soporte, horario, tiempo de respuesta	Soporte 24/7, respuesta en 1 h
REDES	Nombre, proveedor, región, precio	Tipo de red, ancho de banda, IP, conexiones	Red privada + IP pública

9.2 Proveedores y regiones

Un mismo proveedor puede disponer de regiones en Europa, Estados Unidos o Asia, pero no todos los recursos están disponibles en todas las regiones. Por ello, el sistema debe relacionar cada proveedor con las regiones donde ofrece sus servicios y recursos.

La elección de la región puede afectar a la **disponibilidad, precio, latencia y residencia de los datos**. Para nuestro proyecto, las regiones de la **UE** son especialmente importantes cuando se requiere que los datos permanezcan dentro de territorio europeo y se cumplan los requisitos de protección de datos.

De forma general:

- **EU:** menor latencia para usuarios europeos y adecuada para requisitos de residencia de datos en Europa.
- **US:** puede ofrecer diferentes precios, servicios o disponibilidad, pero los datos se alojarán fuera de la UE.
- **Asia:** puede ser interesante para usuarios de esa zona, aunque para clientes europeos normalmente implica mayor latencia.

El sistema permitirá seleccionar **proveedor y región** para configurar los recursos del proyecto y posteriormente calcular su coste.

.

9.3 Proyectos y requerimientos estructurados

Un **Proyecto** representa una necesidad tecnológica de un cliente que debe ser analizada para determinar qué recursos cloud necesita, dónde ubicarlos y qué configuración deben tener. El proyecto puede comenzar con una descripción en texto libre y posteriormente transformarse en requisitos estructurados que permitan seleccionar recursos del catálogo y calcular una configuración y un presupuesto.

Algunos ejemplos de proyectos son:

1. **Academia online:** aplicación Laravel para unos 500 usuarios, con base de datos, almacenamiento y copias de seguridad.

2. **Tienda online:** aplicación web con servidor, base de datos, almacenamiento y monitorización.
3. **Aplicación empresarial:** sistema para una empresa con varios usuarios, base de datos y medidas de seguridad.
4. **Servidor para una API:** servicio web que necesita computación, red, seguridad y monitorización.
5. **Plataforma de inteligencia artificial:** aplicación que necesita recursos de computación IA, almacenamiento y monitorización.

El paso de **texto libre a requisitos estructurados** consiste en analizar la descripción del cliente y extraer información concreta como el tipo de aplicación, número de usuarios, recursos necesarios, capacidad, región, presupuesto, seguridad o copias de seguridad. Estos datos se pueden relacionar posteriormente con los recursos disponibles en el catálogo.

Por ejemplo:

Texto libre:
“Necesito una academia Laravel para unos 500 usuarios en Europa, con copias de seguridad diarias y un presupuesto máximo de 50 € al mes.”

Requisitos estructurados:

- Aplicación: Laravel
- Usuarios: 500
- Región: UE
- Backup: diario
- Presupuesto máximo: 50 €/mes

Los requisitos se pueden clasificar en:

- **Obligatorios:** condiciones que el proyecto debe cumplir. Por ejemplo, región UE, backup diario o presupuesto máximo.
- **Opcionales:** características que pueden añadirse si el presupuesto y la configuración lo permiten. Por ejemplo, monitorización avanzada o un servicio de IA.

Así, el sistema puede utilizar los requisitos estructurados para **buscar recursos compatibles en el catálogo, construir la infraestructura propuesta y calcular su coste**.

9.4 Carrito, presupuesto, pedido y pago simulado

La complejidad del carrito Cloud frente a uno tradicional radica principalmente en la naturaleza dinámica de los recursos basados en la nube. Mientras que un carrito tradicional gestiona productos físicos con stock estático (unidades fijas), un carrito Cloud debe procesar suscripciones, capacidades bajo demanda (escalado de almacenamiento o CPU) y combinaciones complejas de servicios que varían según el tiempo de uso, la región o la concurrencia. Esto exige calcular estructuras de precios variables en tiempo real en lugar de consultar un precio de catálogo fijo.

Para estabilizar este entorno dinámico, **la información que debe congelarse en un presupuesto** es:

- **El precio unitario y la tarifa base** del recurso Cloud en el instante exacto de la solicitud.
- **Las condiciones y límites del servicio** acordados (por ejemplo, ancho de banda contratado, capacidad de almacenamiento máxima o número de usuarios).
- **La vigencia del presupuesto** (fecha de caducidad), garantizando que las fluctuaciones de precios del proveedor Cloud no afecten al cliente durante ese periodo.

Por otro lado, las operaciones críticas que **necesitan atomicidad (propiedad ACID)** para evitar que el sistema quede en un estado inconsistente son:

- **La transición de Carrito a Pedido/Presupuesto:** El bloqueo del recurso simulado y la generación de la orden deben ocurrir simultáneamente; no puede crearse un pedido si el recurso ya no está disponible.
- **El registro del pago y activación:** La confirmación de la pasarela de pago simulada y la actualización del estado de la suscripción a "Activa" deben ejecutarse como una única transacción indivisible. Si falla la actualización del estado, se debe revertir (*rollback*) el cobro simulado.

9.5 Despliegues simulados y auditoría

Por **despliegue simulado** se entiende el proceso en el cual la aplicación recrea el flujo completo de puesta en marcha de un servicio Cloud (como el aprovisionamiento de un servidor, la configuración de una red o el despliegue de una base de datos) dentro de un entorno controlado de pruebas, **sin llegar a realizar llamadas reales a las API de proveedores de infraestructura de pago** (como AWS o Azure). Esto permite validar que la lógica de negocio, las cuotas de usuario y los flujos de autorización funcionan correctamente sin incurrir en costes económicos reales.

Para garantizar la seguridad y la trazabilidad del sistema, las **acciones importantes que deberían quedar registradas para auditoría** (en el log inmutable) son:

- **Autenticación y autorización:** Intentos de inicio de sesión (exitosos y fallidos) y cambios en los roles o privilegios de los actores.
- **Modificación de presupuestos y precios:** Cualquier alteración manual o automatizada de las tarifas base congeladas en un presupuesto activo.
- **Eventos del ciclo de vida del despliegue:** El instante exacto de inicio, confirmación, fallo o cancelación de cada despliegue simulado, asociado directamente al identificador del actor que lo ejecutó.
- **Simulación de transacciones financieras:** El registro de todas las solicitudes y respuestas de la pasarela de pago simulada para evitar fraudes en el historial del cliente.

10. Casos de uso

- ***Aquí documentamos los principales casos de usos hasta el momento:***

Código	Nombre del Caso de Uso	Actor Principal	Estado
CU01	Registro e inicio de sesión de usuario	Usuario Anónimo / Cliente	Revisado
CU02	Consultar catálogo y filtrar recursos Cloud	Cliente / Arquitecto	Revisado
CU03	Gestionar recursos en el carrito de compra	Cliente	Revisado
CU04	Crear y configurar un nuevo proyecto Cloud	Cliente	Revisado
CU05	Introducir requerimientos en lenguaje natural (Procesamiento IA)	Cliente	Revisado
CU06	Diseñar topología y aprobar arquitectura técnica	Arquitecto	Revisado
CU07	Generar y congelar presupuesto formal	Cliente / Arquitecto	Revisado
CU08	Simular pasarela de pago y formalizar pedido	Cliente	Revisado
CU09	Iniciar simulación de despliegue de infraestructura	Arquitecto / Administrador	Revisado
CU10	Consultar logs de auditoría e historial inmutable	Auditor	Revisado
CU11	Administrar usuarios y asignación de roles (RBAC)	Administrador	Revisado
CU12	Mantener catálogo maestro y actualizar tarifas base	Administrador	Revisado

10.1 CU01 — Registro e inicio de sesión de usuario

Campo	Contenido
Código	CU01
Nombre	Registro e inicio de sesión de usuario
Actor principal	Usuario Anónimo / Cliente
Descripción	Permite a un visitante anónimo darse de alta en la plataforma proporcionando sus datos credenciales, o bien autenticarse de forma segura si ya posee una cuenta activa, permitiendo la asignación automática de su rol (RBAC).
Precondiciones	El usuario debe disponer de acceso a la plataforma web y rellenar los campos obligatorios del formulario (email válido, contraseña con longitud mínima). El correo no debe existir previamente en la base de datos para el caso de registros nuevos.
Flujo principal	1. El usuario anónimo accede a la sección de Autenticación/Registro dentro de la interfaz web. 2. El usuario introduce los datos solicitados (Nombre, Correo electrónico y contraseña) en el formulario correspondiente. 3. El sistema valida los datos en el backend, encripta la contraseña usando bcrypt (según políticas RNF03) y almacena el nuevo registro en la entidad USUARIOS asignando por defecto el rol de Cliente. 4. El sistema inicia la sesión de navegación del usuario y habilita el acceso a su cuadro de mando de proyectos.
Resultado / postcondición	El usuario queda correctamente autenticado en el sistema bajo una sesión segura con su respectivo rol activo, permitiendo que cualquier carrito transitorio de su sesión anónima sea gestionado o asociado según los flujos de conflicto establecidos.

10.2 CU02 — Consultar catálogo y filtrar recursos Cloud

Campo	Contenido
Código	CU02
Nombre	Consultar catálogo y filtrar recursos Cloud
Actor principal	Cliente / Arquitecto
Descripción	Permite a los usuarios visualizar el listado maestro de todos los servicios e infraestructura Cloud simulada disponibles en el catálogo, pudiendo aplicar filtros dinámicos basados en el tipo de recurso, proveedor e infraestructura física de las regiones geográficas.
Precondiciones	El sistema debe disponer de un catálogo inicial poblado con registros válidos (entidad CATÁLOGO) y las correspondientes zonas o proveedores (entidad PROVEEDORES / REGIONES) dados de alta.
Flujo principal	1. El usuario accede a la sección del catálogo general de servicios e infraestructuras en la interfaz del sistema. 2. El sistema realiza una consulta a la base de datos local y renderiza en pantalla las tarjetas informativas con las tarifas base de los recursos. 3. El usuario interactúa con la barra de herramientas aplicando un filtro específico (por ejemplo, seleccionando únicamente recursos de "Computación" pertenecientes a la región "UE"). 4. El backend de Laravel recibe la petición filtrada de forma asíncrona, valida los parámetros y devuelve la vista actualizada con los productos compatibles que cumplen el criterio seleccionado.
Resultado / postcondición	El sistema muestra con éxito en pantalla únicamente los recursos e infraestructura Cloud que coinciden con los filtros aplicados por el Cliente o Arquitecto, dejándolos listos para ser seleccionados.

10.3 CU03 — Gestionar recursos en el carrito de compra

Campo	Contenido
Código	CU03
Nombre	Gestionar recursos en el carrito de compra
Actor principal	Cliente
Descripción	Permite al usuario añadir, visualizar, modificar parámetros mínimos o eliminar los diferentes servicios de infraestructura Cloud simulados dentro de su carrito transitorio antes de proceder a la consolidación de un presupuesto o pedido.
Precondiciones	El usuario debe estar navegando por el catálogo de servicios. El sistema debe asegurar que el usuario mantenga un único carrito activo por sesión de navegación para evitar conflictos e inconsistencias (según RF06).
Flujo principal	1. El cliente selecciona un recurso Cloud específico del catálogo y pulsa sobre la opción de añadir al carrito. 2. El sistema intercepta la petición, verifica el tipo de sesión y añade de forma dinámica el ítem actualizando la entidad CARRITO junto con el coste acumulado total estimado. 3. El cliente accede a la vista de desglose de su carrito, desde donde puede reajustar las capacidades del recurso o eliminar elementos innecesarios. 4. El backend calcula matemáticamente los precios vigentes al instante en tiempo real y refresca los elementos en pantalla sin alterar los carritos de otros usuarios.
Resultado / postcondición	El contenido temporal de la infraestructura seleccionada queda modificado de forma coherente en el carrito de compra, recalculando su coste acumulado total y quedando listo para su posterior paso a presupuesto.

10.4 CU04 — Crear y configurar un nuevo proyecto Cloud

Campo	Contenido
Código	CU04
Nombre	Crear y configurar un nuevo proyecto Cloud
Actor principal	Cliente
Descripción	Permite al cliente inicializar un entorno o contenedor lógico de trabajo dentro del sistema, especificando un nombre descriptivo, asignando un presupuesto máximo estimado y definiendo la región base donde planea estructurar sus despliegues.
Precondiciones	El usuario debe estar previamente autenticado en la plataforma con el rol de Cliente activo en el sistema.
Flujo principal	1. El cliente accede a su cuadro de mando y pulsa en la opción "Crear nuevo proyecto". 2. El usuario introduce los datos iniciales de configuración: nombre del proyecto, región geográfica principal (ej: UE) y límite económico mensual sugerido. 3. Al confirmar, el backend de Laravel valida la coherencia de los campos requeridos, crea un registro único inmutable en el módulo de PROYECTOS y lo vincula directamente al identificador de su cuenta de usuario.
Resultado / postcondición	El proyecto cloud se da de alta con éxito en la base de datos vinculado al cliente, quedando habilitado e inicializado para comenzar la inserción de requerimientos o el diseño técnico de arquitecturas

10.5 CU05 — Introducir requerimientos en lenguaje natural (Procesamiento IA)

Campo	Contenido
Código	CU05
Nombre	Introducir requerimientos en lenguaje natural (Procesamiento IA)
Actor principal	Cliente
Descripción	Permite al cliente describir las necesidades de su aplicación en texto libre mediante un campo de formulario (TEXTAREA), delegando en el servicio de IA la traducción inteligente a requerimientos estructurados y la propuesta automatizada de recursos.
Precondiciones	El cliente debe tener un proyecto creado y seleccionado en su sesión de navegación. El servicio backend de la API de IA debe estar plenamente operativo y accesible.
Flujo principal	1. El cliente redacta en lenguaje natural la infraestructura deseada en el cuadro de texto provisto y pulsa en "Procesar con IA". 2. El sistema envía el texto al SERVICIO_IA bajo un prompt estricto, el cual analiza la petición y devuelve una estructura JSON limpia con recursos y regiones teóricas. 3. El backend determinista de Laravel intercepta el JSON, realiza un filtrado estricto contra el CATÁLOGO real (aplicando la regla RN03 e ignorando peticiones inválidas) y carga los recursos válidos en el carrito del usuario.
Resultado / postcondición	El sistema traduce con éxito el texto libre a requerimientos estructurados en la base de datos y añade automáticamente los componentes de infraestructura identificados y validados al carrito de compra activo del cliente

10.6 CU06 — Diseñar topología y aprobar arquitectura técnica

Campo	Contenido
Código	CU06
Nombre	Diseñar topología y aprobar arquitectura técnica
Actor principal	Arquitecto
Descripción	Permite al Arquitecto diseñar de manera técnica los planos lógicos y la disposición de componentes sobre el proyecto del cliente, asociando recursos del catálogo, organizando redes e inyectando las especificaciones de hardware necesarias para su posterior aprobación.
Precondiciones	El proyecto del cliente debe encontrarse en un estado válido de diseño y contar con requerimientos correctamente estructurados y validados (sin estados "Rechazado").
Flujo principal	1. El arquitecto selecciona el proyecto asignado y accede a la sección de modelado de topologías lógicas de infraestructura. 2. El usuario diseña los planos técnicos vinculando las instancias necesarias (servidores, redes lógicas y cortafuegos perimetrales) basándose en las necesidades del cliente. 3. Al finalizar la estructuración, el arquitecto guarda el esquema técnico en el sistema, disparando una verificación automática del backend de Laravel que valida la compatibilidad física y de seguridad de los componentes seleccionados.
Resultado / postcondición	La arquitectura técnica queda consolidada y registrada en la base de datos bajo el estado de "Aprobada", desbloqueando la fase del flujo que permite trasladar la topología diseñada hacia el módulo de presupuestación

10.7 CU07 — Generar y congelar presupuesto formal

Campo	Contenido
Código	CU07
Nombre	Generar y congelar presupuesto formal
Actor principal	Cliente / Arquitecto
Descripción	Permite consolidar los recursos Cloud del carrito o de la arquitectura aprobada en un documento presupuestario formal, congelando los precios unitarios y tarifas vigentes en ese instante exacto para evitar que las fluctuaciones del mercado afecten al cliente durante su validez.
Precondiciones	El proyecto debe contar con una arquitectura en estado "Aprobada por el Cliente" o el carrito de compra debe disponer de ítems válidos y un coste calculado.
Resultado / postcondición	La transacción simulada queda registrada con éxito, consolidando el carrito/presupuesto en un pedido en firme en la base de datos y desbloqueando el entorno para la simulación técnica de los despliegues de infraestructura.

10.8 CU08 — Simular pasarela de pago y formalizar pedido

Campo	Contenido
Código	CU08
Nombre	Simular pasarela de pago y formalizar pedido
Actor principal	Cliente
Descripción	Permite al cliente realizar la simulación de una transacción financiera a través del entorno de pruebas (Sandbox) de la pasarela de pago (Stripe/PayPal), validando el resultado para convertir de forma atómica el presupuesto en un pedido formal e inmutable.
Precondiciones	El usuario debe estar autenticado como Cliente. Debe existir un presupuesto vigente (no expirado) en estado "emitido" y la pasarela de pruebas externa debe estar disponible de forma síncrona.
Flujo principal	1. El cliente pulsa "Proceder al pago" en un presupuesto activo e introduce las credenciales de una tarjeta de crédito ficticia en el formulario seguro de la pasarela simulada. 2. El sistema envía la solicitud al entorno Sandbox, el cual valida la operación devolviendo una respuesta de confirmación exitosa (o un error controlado si no hay saldo o hay timeout). 3. Laravel procesa la respuesta bajo una única transacción de base de datos (ACID): cambia el estado del presupuesto a "Aceptado", genera el registro en la entidad PEDIDOS en estado "Procesado" y vacía por completo el contenido del carrito activo.
Resultado / postcondición	La transacción simulada queda registrada con éxito, consolidando el carrito/presupuesto en un pedido en firme en la base de datos y desbloqueando el entorno para la simulación técnica de los despliegues de infraestructura.

10.9 CU09 — Iniciar simulación de despliegue de infraestructura

Campo	Contenido
Código	CU09
Nombre	Iniciar simulación de despliegue de infraestructura
Actor principal	Sistema / Administrador (o Arquitecto)
Descripción	Permite disparar de forma automática o manual el proceso secuencial que emula el aprovisionamiento físico y la puesta en marcha de los servidores, redes y recursos lógicos que componen el pedido formalizado del cliente.
Precondiciones	El proyecto debe estar asociado a un pedido que se encuentre estrictamente en estado "Procesado" o "Pagado" tras haber superado con éxito la pasarela de pago.
Flujo principal	1. El usuario accede al panel de control del proyecto abonado y pulsa sobre la acción "Iniciar Simulación de Despliegue". 2. El backend de Laravel cambia el estado de la infraestructura a "En proceso de despliegue" e inicia un hilo o tarea en segundo plano (Queue/Job) que ejecuta de manera determinista la simulación cronometrada de cada componente. 3. El sistema actualiza en tiempo real la interfaz gráfica del cliente mediante eventos, mostrando el progreso detallado (ej: Creando red lógica... Levantando el servidor web... Configurando DNS...) hasta completar el árbol de la topología. 4. Al finalizar la secuencia sin errores, el motor de la plataforma consolida la infraestructura simulada y abre los accesos lógicos de monitorización.
Resultado / postcondición	La infraestructura del proyecto cambia su estado definitivo a "Desplegado" o "Activo", quedando completamente operativa dentro del simulador para que el cliente pueda realizar su posterior seguimiento y auditoría.

10.10 CU10 — Monitorizar métricas y alertas del sistema

Campo	Contenido
Código	CU10
Nombre	Monitorizar métricas y alertas del sistema
Actor principal	Cliente / Arquitecto
Descripción	Permite a los usuarios visualizar gráficamente el rendimiento, consumo simulado (CPU, RAM, almacenamiento) y tráfico de red de sus recursos Cloud desplegados, así como recibir notificaciones automáticas o alertas en caso de anomalías o sobrecostes.
Precondiciones	El proyecto seleccionado debe tener su infraestructura en estado "Desplegado" o "Activo" (según las postcondiciones del CU09).
Flujo principal	1. El usuario accede al cuadro de mando del proyecto activo y entra en la pestaña de "Monitorización y Métricas". 2. El sistema realiza una consulta al backend de Laravel, el cual genera y recupera los datos históricos simulados de uso para renderizarlos en gráficos interactivos en tiempo real. 3. El backend verifica de forma paralela si algún parámetro supera los umbrales configurados por el usuario (por ejemplo, consumo de RAM superior al 90% o presupuesto diario excedido). 4. Si se detecta una anomalía, el sistema dispara un evento visual en el panel y envía una alerta controlada (por correo electrónico o notificación interna) al Cliente.

10.11 CU11 — Administrar y mantener el catálogo de recursos Cloud

Campo	Contenido
Código	CU11
Nombre	Administrar y mantener el catálogo de recursos Cloud
Actor principal	Administrador
Descripción	Permite al personal interno con privilegios de gestión añadir nuevos servicios de infraestructura al catálogo maestro, actualizar los precios por hora/mes de los recursos existentes o modificar la disponibilidad física de las regiones lógicas.
Precondiciones	El usuario debe haber iniciado sesión en la plataforma y contar explícitamente con el rol de Administrador asignado mediante el sistema de control de accesos (RBAC).
Flujo principal	1. El administrador accede al panel de control interno de gestión de la plataforma y selecciona la sección "Mantenimiento del Catálogo". 2. El usuario modifica los parámetros de un recurso (por ejemplo, actualizando la tarifa base de una instancia de cómputo) o rellena el formulario para dar de alta un nuevo tipo de base de datos simulada. 3. Al guardar los cambios, el backend de Laravel valida la integridad de los datos ingresados y actualiza la entidad CATÁLOGO de manera persistente en la base de datos global.
Resultado / postcondición	Las modificaciones del catálogo se aplican de manera inmediata en el sistema, quedando disponibles de forma transparente para las futuras búsquedas de los clientes (CU02) y para el filtrado del procesador de IA (CU05).

10.12 CU12 — Consultar histórico y auditoría del proyecto

Campo	Contenido
Código	CU12
Nombre	Consultar histórico y auditoría del proyecto
Actor principal	Cliente / Arquitecto
Descripción	Permite a los usuarios visualizar la línea de tiempo completa y el registro histórico de un proyecto seleccionado, incluyendo cambios de estado, presupuestos asociados, logs de despliegues y peticiones procesadas por la IA.
Precondiciones	El usuario debe estar autenticado y ser el propietario del proyecto (o tener rol de Arquitecto/Administrador con acceso concedido). El proyecto debe registrar actividad previa en la base de datos.
Flujo principal	1. El usuario accede a la sección de sus proyectos y selecciona la pestaña "Historial y Auditoría". 2. El sistema realiza una consulta agregada a las tablas de logs, pedidos y estados vinculadas a ese identificador único de proyecto. 3. Laravel ordena cronológicamente los hitos y los renderiza en una línea de tiempo visual, permitiendo al usuario descargar los justificantes o contratos de pedidos anteriores.
Resultado / postcondición	El usuario visualiza con éxito el registro de auditoría de su infraestructura cloud simulada, garantizando la trazabilidad de todas las operaciones realizadas.

11. Modelo conceptual de datos

11.1 Entidades identificadas

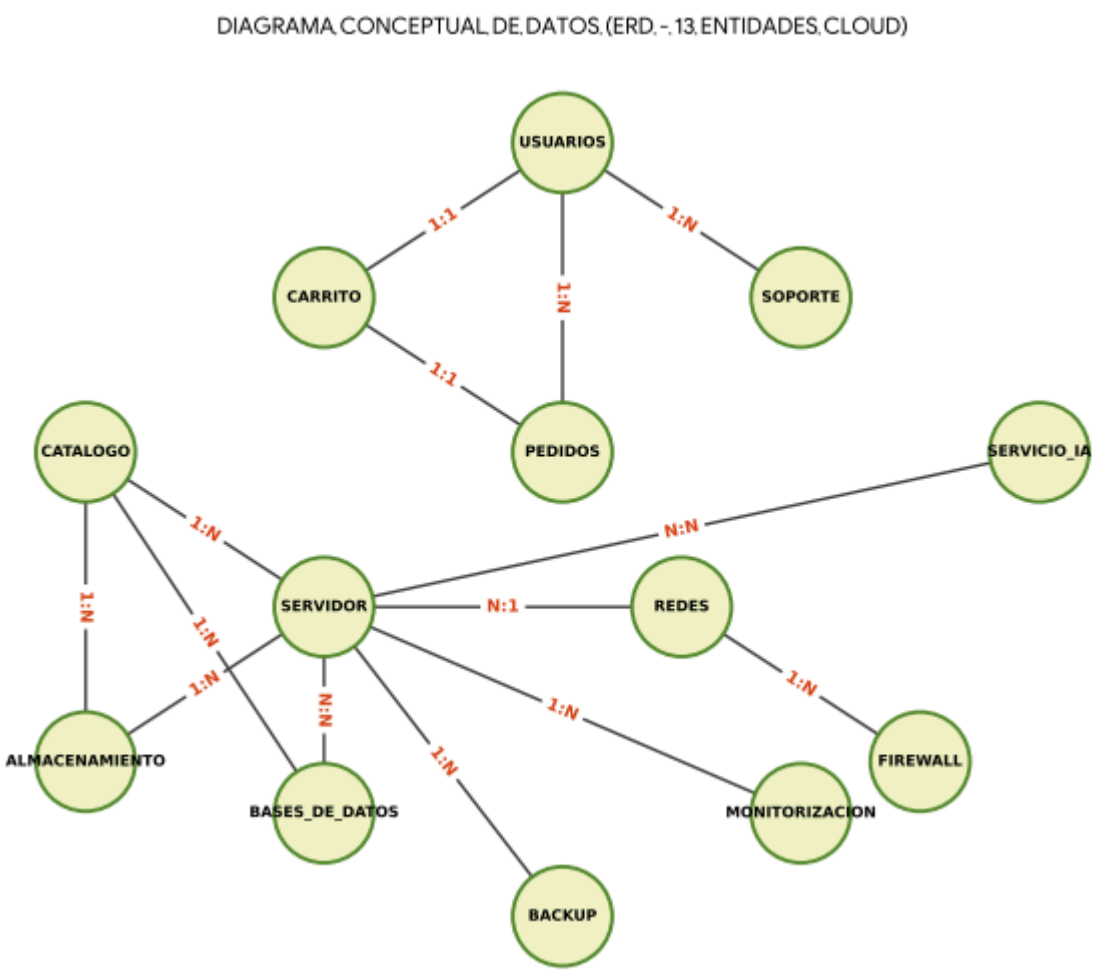
Entidad	Responsabilidad	Datos principales	Observaciones
USUARIOS	Gestionar cuentas de acceso, credenciales de clientes y el control de roles técnicos.	id_usuario, nombre, email, password_hash, rol.	Soporta el acceso al carrito y la gestión de pedidos.
CATÁLOGO	Ofrecer el listado maestro de todos los recursos e infraestructura Cloud simulada disponibles.	id_catalogo, nombre_recurso, categoria, precio_base.	Es el punto de consulta para armar configuraciones.
CARRITO	Almacenar temporalmente los recursos e infraestructura seleccionados por un usuario antes de comprar.	id_carrito, id_usuario, fecha_creacion, coste_acumulado.	Está ligado a la sesión activa del usuario.
PEDIDOS	Formalizar la orden de contratación de la infraestructura y procesar el flujo del pago.	id_pedido, id_usuario, fecha_pedido, estado_pedido, total.	Requiere atomicidad y validación mediante esquema PDF.
SERVIDOR	Modelar las instancias de computación virtual simuladas (tipo CPUs/RAM virtuales).	id_servidor, nombre_instancia, vCPUs, memoria_RAM, sistema_operativo.	Recurso núcleo de cómputo en la arquitectura.
ALMACENAMIENTO	Gestionar los volúmenes de disco duro virtuales y almacenamiento de archivos en la nube.	id_almacenamiento, tipo_disco (SSD/HDD), capacidad_GB, tasa_transferencia.	Almacena los datos persistentes de los servidores.

BASES_DE_DATOS	Administrar los motores de bases de datos simulados (relacionales o NoSQL).	id_db, motor (MySQL/PostgreSQL), capacidad_almacenamiento, max_conexiones.	Infraestructura optimizada para la persistencia del sistema.
REDES	Configurar el direccionamiento lógico, subredes y topología de interconexión virtual.	id_red, rango_CIDR, zona_disponibilidad, id_proyecto.	Estructura el canal de comunicación de los servidores.
FIREWALL	Definir las reglas de seguridad perimetral para filtrar el tráfico entrante y saliente.	id_firewall, id_red, puerto_origen, puerto_destino, accion (Allow/Deny).	Asegura que el entorno de red esté protegido.
BACKUP	Gestionar las copias de seguridad automáticas y políticas de recuperación ante desastres.	id_backup, id_recurso_origen, frecuencia, fecha_ultima_copia, retencion_dias.	Protege los datos de almacenamiento y bases de datos.
MONITORIZACIÓN	Registrar el rendimiento, uso de recursos y alertas técnicas del entorno Cloud.	id_metrica, id_recurso, uso_cpu_porcentaje, alertas_activas, marca_tiempo.	Ofrece visibilidad del estado de la infraestructura.
SERVICIO_IA	Integrar servicios cognitivos o modelos de inteligencia artificial bajo demanda en el sistema.	id_ia_servicio, tipo_api (Vision/NLP), peticiones_max_min, coste_por_token.	Módulo avanzado de procesamiento inteligente de datos.
SOPORTE	Canalizar incidencias, tickets de ayuda y asistencia técnica sobre la infraestructura.	id_ticket, id_usuario, asunto, descripcion_problema, estado_ticket.	Permite la comunicación cliente-técnico ante fallos.

11.2 Relaciones principales

Entidad A	Relación	Entidad B	Cardinalidad / explicación
USUARIOS	Gestiona / Posee	**CARRITO**	**1:1** (Un usuario tiene un único carrito activo por sesión).
USUARIOS	Realiza / Compra	**PEDIDOS**	**1:N** (Un usuario puede realizar muchos pedidos a lo largo del tiempo).
CATÁLOGO	Contiene	**SERVIDOR**	**1:N** (El catálogo ofrece diferentes tipos de instancias de servidores virtuales).
CATÁLOGO	Contiene	**ALMACENAMIENTO**	**1:N** (El catálogo lista las opciones de discos duros SSD o HDD configurables).
CATÁLOGO	Contiene	**BASES_DE_DATOS**	**1:N** (El catálogo ofrece distintos motores de bases de datos simulados).
SERVIDOR	Se conecta a	**REDES**	**N:1** (Muchos servidores virtuales pueden estar integrados dentro de una misma red lógica).
REDES	Es protegida por	**FIREWALL**	**1:N** (Una red puede tener asociadas varias reglas de filtrado perimetral).
SERVIDOR	Almacena datos en	**ALMACENAMIENTO**	**1:N** (Un servidor puede tener montados uno o varios discos virtuales de almacenamiento).
SERVIDOR	Utiliza	**BASES_DE_DATOS**	**N:N** (Muchos servidores pueden consultar muchas bases de datos, y viceversa).
SERVIDOR	Es respaldado por	**BACKUP**	**1:N** (Un servidor o recurso puede generar múltiples copias de seguridad programadas).
SERVIDOR	Es monitorizado por	**MONITORIZACIÓN**	**1:N** (Un servidor genera un flujo continuo de múltiples métricas de rendimiento).
SERVIDOR	Consume	**SERVICIO_IA**	**N:N** (Muchos servidores pueden realizar llamadas de computación a diferentes servicios de IA).
USUARIOS	Abre / Reporta	**SOPORTE**	**1:N** (Un usuario puede abrir múltiples tickets de soporte ante incidencias técnicas).
PEDIDOS	Incluye ítems de	**CARRITO**	**1:1** (Un pedido se consolida a partir de la conversión de un único carrito de compra).

11.3 Diagrama conceptual



11.3 Diagrama Conceptual de Datos ERD entidades CLOUD

11.4 Decisiones abiertas del modelo

A continuación, se enumeran las decisiones de modelado que todavía no están completamente cerradas y que requieren una revisión detallada durante la **UT2**:

- **Relación N:N entre SERVIDOR y SERVICIO_IA**:
  Actualmente se plantea una relación de muchos a muchos. Debemos evaluar en UT2 si esta relación requiere una tabla intermedia que almacene atributos específicos (como los prompts enviados, los tokens consumidos o el timestamp de la consulta) o si el SERVICIO_IA solo actúa como una función externa y no como una entidad persistente clásica.
- **Dependencia jerárquica del CATÁLOGO**:
  El diagrama muestra que el CATALOGO tiene relaciones *1:N* directas hacia SERVIDOR, ALMACENAMIENTO y BASES_DE_DATOS. Se debe revisar en UT2 si es más eficiente unificar todos los recursos en una sola entidad genérica "Producto/Recurso" mediante herencia (tipo ISA) o si se mantiene la separación actual, lo cual complicaría la lógica del CARRITO.

- **Vinculación de la MONITORIZACIÓN y BACKUP con los recursos**:
  En el diagrama actual, MONITORIZACION y BACKUP están conectados únicamente a SERVIDOR (*1:N*). Se debe decidir si estos servicios también deberían poder aplicarse de forma independiente a las BASES_DE_DATOS o al ALMACENAMIENTO sin necesidad de que el cliente contrate un servidor completo.
- **Persistencia del CARRITO**:
  La relación es *1:1* entre USUARIOS y CARRITO. Falta definir si el carrito se almacenará de forma persistente en la base de datos (permitiendo recuperar la sesión en diferentes dispositivos) o si se gestionará temporalmente en la sesión del lado del cliente (LocalStorage/Redis), lo que eliminaría la entidad del modelo relacional final.
- **Acoplamiento de REDES y FIREWALL**:
  La relación *1:N* entre REDES y FIREWALL asume que un cortafuegos pertenece a una única red. Se debe validar si en entornos de producción reales un FIREWALL puede aplicar reglas perimetrales a múltiples redes o subredes simultáneamente

.

12. Arquitectura Laravel inicial

- ***Explicamos la función que tendrá cada grupo de elementos de Laravel.***

Elemento	Función prevista	Ejemplo dentro del proyecto
routes/	Definir los puntos de entrada (endpoints) de la aplicación web y la API, mapeando las URLs con sus respectivos controladores y middlewares.	Route::post('/ai/process', [AiController::class, 'process']) para procesar el lenguaje natural o routes/api.php para la pasarela de pago.
Controllers/	Actuar como intermediarios ligeros que reciben las peticiones HTTP, invocan la lógica de negocio y devuelven una respuesta (vista o JSON).	CartController.php para recibir la acción de añadir un recurso o OrderController.php para gestionar la pasarela de pago.
Models/	Representar las tablas de la base de datos mediante clases de Eloquent, gestionando la persistencia de datos y las relaciones (1:N, N:M).	El modelo Project.php que tiene una relación belongsTo(User::class) o el modelo Resource.php para el catálogo.
Resources/	Capa de transformación que convierte los modelos y colecciones de Eloquent en estructuras JSON limpias y seguras para la API o el frontend.	ProjectResource.php para devolver al cliente los detalles técnicos del proyecto y el árbol de topología formateado.
Enums/	Tipos de datos que limitan los valores posibles de un campo a	ProjectStatus::class con los estados fijos: Borrador,

	un conjunto cerrado de constantes, garantizando la consistencia del sistema.	Aprobado, EnProcesoDespliegue o Desplegado.
Requests/	Clases especializadas en encapsular las reglas de validación y autorización de los datos de entrada antes de que lleguen al controlador.	StorePromptRequest.php para validar que el campo textarea del lenguaje natural no esté vacío y no supere el límite de caracteres.
Services/	Capa de abstracción donde se almacena la lógica de negocio compleja y pesada, liberando de carga a los controladores y modelos.	AiService.php para conectar con la API de IA externa o PaymentService.php para gestionar los reintentos y tokens de Stripe.
Policies/ (futuro)	Clases encargadas de centralizar las reglas de autorización (RBAC) sobre un modelo, determinando si un usuario puede realizar una acción.	ProjectPolicy.php para verificar mediante el método view si el Cliente actual es verdaderamente el propietario del proyecto que intenta auditar.
Ai/Agents/ (futuro)	Clases avanzadas destinadas a estructurar el comportamiento, prompts del sistema y el flujo lógico de los agentes conversacionales de IA.	InfrastructureAgent.php para configurar el prompt maestro que fuerza a la IA a devolver un JSON e

12.1 Separación de responsabilidades

En un proyecto como **Cloud 9 Systems**, que tiene que conectar con una IA para traducir texto y

simular pagos con una pasarela, el código se puede volver un caos muy rápido si lo metemos todo en el mismo sitio. Si programamos toda la lógica dentro de los controladores, acabaremos teniendo archivos gigantescos (el típico antipatrón *Fat Controller*). Esto hace que el código sea dificilísimo de leer, que sea imposible encontrar los fallos y que, si cambiamos algo en el carrito, se rompa la parte de la IA.

Para evitarlo, vamos a aplicar a rajatabla la **separación de responsabilidades**, haciendo que los controladores sean "delgados" y se limiten a redirigir el tráfico. Cada herramienta de Laravel hará solo su trabajo:

- **Validar los datos (Form Requests):** El controlador no se va a manchar las manos comprobando si los campos obligatorios están vacíos. Si el usuario envía el formulario de la IA, una clase StorePromptRequest validará que el texto sea correcto. Si falta algo, Laravel rebota la petición antes de que llegue siquiera al controlador.
- **La lógica pesada a los Servicios (Services):** Los controladores solo deben recibir la petición y llamar a un servicio. Toda la lógica compleja (conectar con la API externa de la IA, calcular los precios con el IVA para el presupuesto o procesar el pago simulado) se programa en archivos separados dentro de la carpeta Services. Así, si mañana queremos usar esa misma lógica en otra parte de la web, solo tenemos que llamar al servicio sin repetir código.
- **Aislar la IA (Agents):** Todo lo que tenga que ver con preparar el prompt para la IA o procesar el JSON que nos devuelve no se mete en cualquier lado. Lo dejamos en su propia carpeta de agentes. Así, si el año que viene cambiamos de modelo de IA, solo retocamos ese archivo y el resto de la web sigue funcionando igual.
- **Limpiar las respuestas (API Resources):** No vamos a enviar los datos de los modelos de la base de datos tal cual al frontend, porque podríamos filtrar información sin querer. Usamos los Resources para empaquetar y formatear el JSON con los campos exactos que necesita la vista.

En resumen: dividimos el proyecto en piezas pequeñas para que el código esté ordenado, sea fácil de entender por cualquiera del grupo y podamos programar cada uno una parte sin pisarnos las funciones.

12.2 Diagrama de arquitectura inicial

12.2 Diagrama de arquitectura inicial

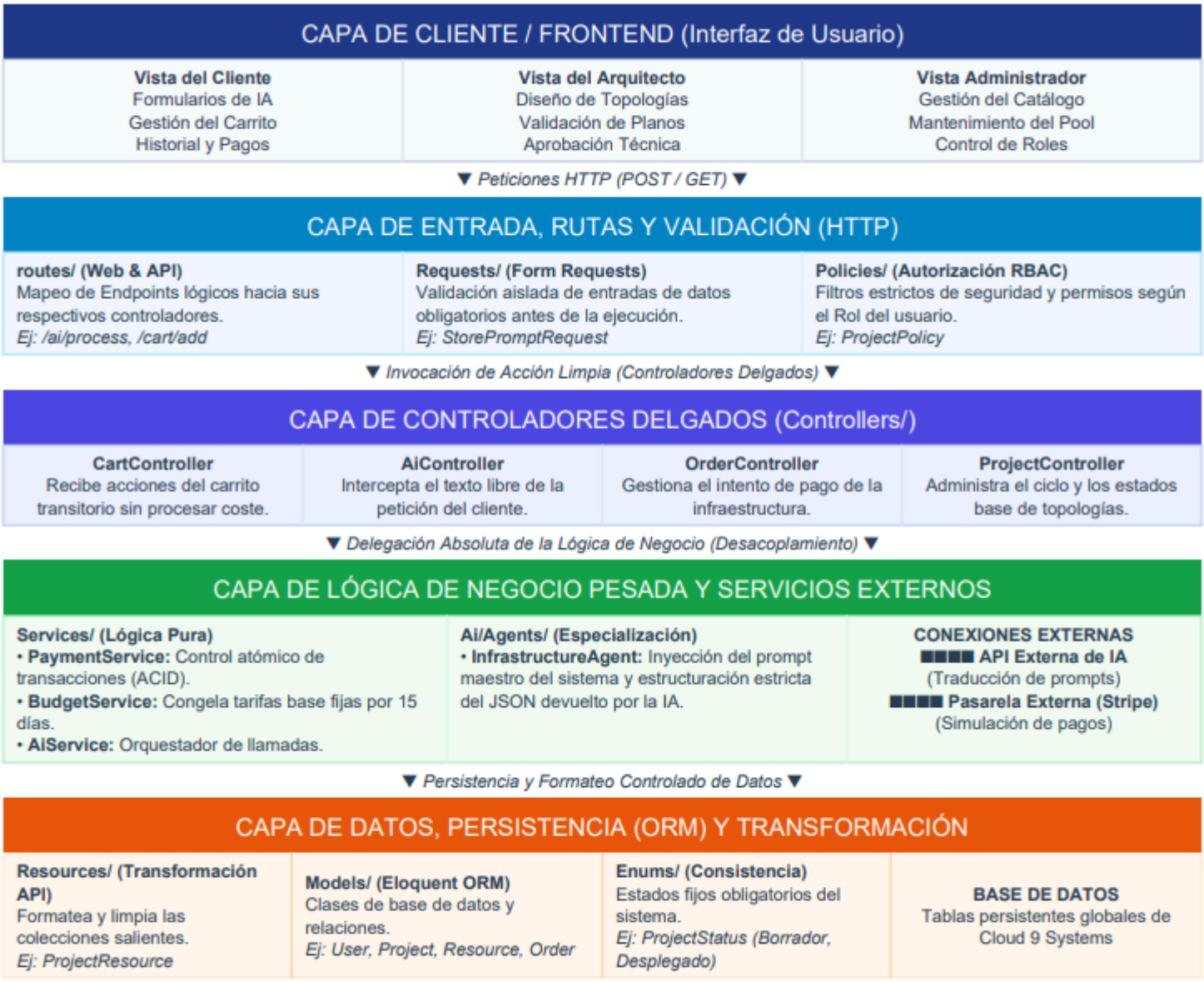


Figura 12.1. Estructura de arquitectura modular y flujo de responsabilidades de Cloud 9 Systems en Laravel.

13. Análisis del uso futuro de Inteligencia Artificial

La arquitectura del sistema establece una separación estricta entre la capa generativa y la capa de negocio: la IA actúa como asistente de recomendación de productos, mientras que el backend en Laravel se mantiene como autoridad para determinar y validar compatibilidades, aplicar ciertas reglas de seguridad e inclusive calcular costes reales.

13.1 Funciones previstas de la IA

- Traducción del lenguaje natural a requerimientos técnicos.
- Recomendación de arquitectura óptima.
- Asistente de optimización de costes y recursos.

13.2 Entradas y salidas previstas

Entrada disponible para IA	Salida / propuesta esperada	Validación posterior en Laravel
Texto libre del usuario con la descripción del proyecto y sus necesidades	Estructura JSON con tipos de recursos recomendados, cantidades y regiones teóricas.	Verificación de la existencia de los productos en la base de datos local y comprobación del stock
Restricciones del usuario, por ejemplo en cuanto a presupuestos o ubicación en la UE	Selección de proveedores y regiones que cumplan con la soberanía de datos	Validación de las reglas de negocio de región y cálculo matemático estricto del coste mensual
Selección previa de componentes en el carrito o borrador de arquitectura	Sugerencia de componentes faltantes o recomendados	Comprobación backend de dependencias de red, compatibilidad entre versiones y reglas de seguridad obligatorias.

13.3 Límites de la IA

- Aunque el asistente de IA actúa como un facilitador inteligente para el diseño de topologías y optimización de recursos, su autonomía está estrictamente limitada. La IA opera exclusivamente en un entorno consultivo y de recomendación; carece de autoridad ejecutiva sobre el sistema.

Por seguridad, integridad y cumplimiento normativo, se establecen los siguientes límites:

- **Precios definitivos y presupuestación:** La IA puede estimar costes, pero nunca emitirá

tarifas comerciales definitivas ni altera los multiplicadores de coste del catálogo. Los presupuestos reales se calculan exclusivamente de forma determinista mediante la lógica del servidor de Laravel.

- **Autorización y gestión de accesos (RBAC):** El asistente no tiene potestad para otorgar, modificar o revocar roles de usuario, ni para alterar los permisos de seguridad del sistema.
- **Reglas de seguridad y firewall:** La IA no puede saltarse las directrices de seguridad de la red simulada. Cualquier diseño generado debe pasar por las validaciones rígidas del código backend antes de ser procesado.
- **Operaciones irreversibles**: Acciones críticas como el borrado o archivado de proyectos, la eliminación de usuarios o la cancelación de pedidos quedan totalmente fuera del alcance de la IA. Requieren siempre la confirmación manual y explícita de un actor humano calificado (Cliente, Arquitecto o Administrador).

13.4 Riesgos de la integración

La incorporación de un modelo de lenguaje (LLM) para la asistencia en la configuración Cloud introduce vulnerabilidades y retos operativos que el sistema debe mitigar:

- **Alucinaciones:** El modelo puede generar configuraciones técnicas inexistentes, topologías físicamente incompatibles o inventar parámetros de hardware ficticios que no se encuentran en el catálogo real.
- **Datos inválidos:** Riesgo de que la IA responda con estructuras de datos corrompidas o formatos JSON malformados que puedan provocar excepciones o caídas en los controladores de Laravel al intentar procesar la sugerencia.
- **Coste económico:** Las peticiones continuas a la API del proveedor de IA (como OpenAI o Anthropic) representan un gasto variable que puede dispararse sin un control de cuotas o un sistema de almacenamiento en caché (*caching*) para consultas repetitivas.
- **Privacidad de los datos**: Peligro de filtración de información sensible del proyecto o de los clientes al enviar datos contextuales hacia servidores externos para su procesamiento.
- **Prompt Injection (Inyección de instrucciones):** Vulnerabilidad donde un usuario malintencionado manipula las entradas de texto del asistente para forzar a la IA a ignorar sus restricciones del sistema, revelar logs internos o simular comportamientos no autorizados.
- **Dependencia del proveedor (Vendor Lock-in)**: Alta subordinación a la disponibilidad, cambios de tarifas o modificaciones en las políticas de uso de la API del proveedor de IA elegido, lo que podría paralizar la funcionalidad si no se implementa una arquitectura flexible de abstracción.
- Necesidad de trazabilidad: Debido a la naturaleza de "caja negra" de la IA, cada recomendación de diseño estructural debe quedar estrictamente registrada en el módulo de Auditoría. Esto es indispensable para analizar qué datos originaron una mala sugerencia técnica en caso de fallos.

14. Riesgos técnicos, seguridad y privacidad

ID	Riesgo	Probabilidad	Impacto	Medida prevista
R01	Exposición accidental de credenciales, tokens o claves API en el repositorio	Media	Alto	Inclusión estricta de .env en .gitignore, distribución de .env.example con claves genéricas y revisión obligatoria de Pull Requests antes de cada integración.
R02	Pérdida o corrupción de información del proyecto durante el desarrollo	Baja	Alto	Control de versiones en Git y uso de migraciones y seeders (php artisan migrate:fresh --seed) para reconstruir el estado de la base de datos local.
R03	Errores en la configuración de recursos Cloud que generen presupuesto incorrecto	Media	Medio	Pruebas unitarias sobre el motor de cálculo de costes y uso de transacciones atómicas de base de datos para garantizar la precisión de los totales.
R04	Accesos no autorizados a funcionalidades según el rol del usuario	Media	Alto	Control de acceso basado en roles y permisos (RBAC) mediante middleware, Gates y Policies nativos de Laravel.
R05	Errores en las reglas de compatibilidad entre recursos Cloud	Media	Medio	Validación determinista en el backend mediante servicios de Laravel que comprueben dependencias de red, firewalls y

				regiones antes de guardar la infraestructura.
R06	Vulnerabilidades en la aplicaciones Laravel o en sus dependencias	Media	Alto	Ejecución periódica de análisis de seguridad con composer audit y npm audit junto con la actualización de paquetes a versiones estables.
R07	Uso incorrecto o exposición de información introducida por los usuario	Media	Alto	Validaciones mediante Form Requests, uso del ORM Eloquent contra SQL Injection, cifrado de contraseñas y protecciones CSRF y XSS.
R08	Recomendaciones incorrectas generadas posteriormente por la IA	Media	Medio	Validación estricta de las sugerencias JSON de la IA contra el catálogo real mediante la lógica de negocio de Laravel antes de mostrarlas al cliente.
R09	Cambios de código que introduzcan errores en funcionalidades existentes	Media	Medio	Desarrollo aislado en ramas feature/..., ejecución de pruebas de regresión y aprobación obligatoria de revisiones de código en GitHub.
R10	Pérdida de trazabilidad sobre quién realizó una modificación	Media	Medio	Implementación de un sistema de registro de auditoría (Audit Log) que almacene usuario, acción, IP, fecha y valores anteriores/nuevos.

14.1 Seguridad mínima del repositorio

***Para garantizar la integridad del código fuente, proteger la información confidencial y prevenir accesos no autorizados, se establecen las siguientes medidas de seguridad obligatorias en el repositorio del proyecto**:*

Gestión de credenciales y variables de entorno (.gitignore y .env.example):

- El archivo .env con las contraseñas reales de base de datos, claves de encriptación de Laravel y tokens de servicios externos está estrictamente excluido del control de versiones mediante el archivo .gitignore.

- Se mantiene actualizado el archivo .env.example en el repositorio con las claves de configuración necesarias en formato genérico o de plantilla, sirviendo de guía segura para que cada desarrollador configure su propio entorno local.

- Está totalmente prohibido incluir hardcoded (directamente en el código) API Keys, credenciales o secretos en cualquier commit o comentario.

Protección de ramas y política de integración (Branch Protection Rules):

-Las ramas principales main y develop están protegidas contra publicaciones directas (git push bloqueado).

-Todo cambio en el código debe enviarse mediante una rama de característica (feature/...) y fusionarse exclusivamente a través

de una Pull Request (PR).

-Las PR hacia develop o main requieren la revisión y aprobación obligatoria de al menos otro integrante del equipo antes de ser fusionadas, garantizando la supervisión cruzada de código y documentación.

Auditoría periódica de dependencias de terceros:

- Se ejecutan análisis automáticos de vulnerabilidades utilizando los comandos composer audit y npm audit antes de cada entrega o integración relevante en la rama principal.

- Ningún paquete o librería externa se actualizará o añadirá al proyecto sin verificar previamente su procedencia, licencia y ausencia de fallos críticos de seguridad conocidos.

Identificación y trazabilidad de colaboradores:

- Todos los integrantes deben configurar su cliente de Git local con el mismo correo electrónico y nombre de usuario vinculado a su cuenta de GitHub, garantizando la autoría e identificabilidad de cada cambio.

-El acceso al repositorio remoto se gestiona mediante autenticación segura basada en claves SSH o Personal Access Tokens (PAT) con caducidad definida.

Protocolo de actuación ante filtración accidental (Plan de Contingencia):

- Si se detecta un token o clave privada subida accidentalmente al historial de Git, se procederá inmediatamente a revocar e invalidar dicha credencial en el servicio de origen.

- Se purgará el historial de commits mediante herramientas de limpieza de Git (ej. git filter-repo) y se forzará la actualización del repositorio remoto para eliminar cualquier rastro del secreto expuesto.

15. Planificación inicial

La planificación es preliminar y podrá evolucionar. Debe identificar bloques de trabajo, dependencias y entregables esperados.

15.1 Backlog inicial de grandes tareas

ID	Tarea / épica	Prioridad	Dependencias
EP01	Preparación del Entorno y Repositorio: Configuración de Ubuntu, PHP, Laravel, estructura de carpetas Git, .env.example y guía docs/entorno.md.	Alta	Ninguna
EP02	Modelo de Datos y Autenticación: Creación de migraciones, modelos de dominio (Proyecto, Recurso Cloud, Proveedor, Tarifa), esquema ER y sistema de usuarios/roles.	Alta	EP01
EP03	Gestión de Proyectos y Requerimientos: Desarrollo del formulario de captura de necesidades, restricciones de soberanía regional (UE) y panel de proyectos.	Alta	EP02
EP04	Motor de Presupuestos y Carrito: Lógica de cálculo de costes recurrentes y únicos, congelación de ofertas, carrito de compra y alerta presupuestaria al superar 50 €/mes.	Media	EP03
EP05	Integración de IA y Recomendaciones: Asistente para	Media	EP03, EP04

	traducción de lenguaje natural a recursos del catálogo con validación en Laravel.		
EP06	Integración de IA y Recomendaciones: Asistente para traducción de lenguaje natural a recursos del catálogo con validación determinista previa en Laravel. Simulados y Auditoría: Proceso de simulación de pago, cambio de estado de pedidos, cuadro de mando de recursos desplegados y log de auditoría.	Media	EP04, EP05

16. Conclusiones

16.1 Estado del proyecto al finalizar UT1

Al concluir la UT1, **el proyecto ha consolidado exitosamente su fase de abstracción, diseño arquitectónico y modelado conceptual**. Se han definido con precisión los 12 módulos funcionales del sistema, sus flujos de datos, reglas de negocio rígidas y los límites del asistente de Inteligencia Artificial.

El proyecto está plenamente preparado para comenzar la implementación en la UT2 debido a que:

1. Contamos con un mapa relacional claro de la base de datos (clases y requerimientos).
2. Se ha establecido una arquitectura de software limpia y desacoplada (Service Pattern) en Laravel.
3. Se han mitigado de antemano los riesgos de seguridad y de integración con la IA, dejando la mesa lista para una codificación fluida y sin ambigüedad

16.2 Principales decisiones tomadas

- **Adopción del Service Pattern en Laravel:** Se decidió rechazar el uso de controladores saturados ("Fat Controllers"), delegando el 100% de la lógica de negocio a Clases de Servicio independientes para garantizar la escalabilidad y mantenibilidad del código.
- **Aislamiento y Rol Consultivo de la IA:** Se determinó de manera unánime que el asistente de IA funcione estrictamente como un entorno de recomendación técnica, bloqueando su acceso a operaciones críticas como la alteración de precios, la gestión de accesos RBAC o

modificaciones directas en la base de datos.
- **Estrategia Inmutable de Logs (Append-Only):** Para blindar el sistema contra fraudes, se definió que el módulo de Auditoría/IA maneje una base de datos de solo lectura e inserción, asegurando que ningún usuario (ni siquiera el Administrador) pueda alterar el historial de acciones.
- **Cifrado Obligatorio en Servidor:** Se estableció el hashing estricto de credenciales en el backend como paso mandatorio previo a cualquier interacción con otros módulos.

16.3 Aspectos que deberán revisarse en UT2

- **Estrategia de Caching para la API de IA:** Evaluar herramientas como Redis dentro de Laravel para almacenar en caché las solicitudes frecuentes del asistente Cloud y controlar así los costes económicos de la API.
- **Validación de Esquemas JSON de la IA:** Diseñar los Form Requests o validadores específicos en Laravel para asegurar que las respuestas estructuradas que devuelva la IA no rompan el flujo de la aplicación.
- **Simulación de Latencias y Costes Regionales:** Testear a fondo el comportamiento de los disparadores automáticos en Laravel cuando una región del catálogo se marque como "Inactiva".
- **Configuración del Entorno de Integración Continua (CI/CD):** Definir las herramientas automáticas que auditarán los commits bajo la norma *Conventional Commits* antes de permitir los *Pull Requests*.

16.4 Reflexión del equipo

Durante esta primera fase, uno de los mayores retos ha sido **la coordinación del equipo y lograr que toda la información encajara bien**. Diseñar la base de datos y la estructura del backend requiere muchísimo más tiempo del que parece. A medida que avanzábamos y surgían nuevos detalles, integrarlos con lo que ya teníamos hecho se volvía complicado y nos obligaba a reorganizarnos constantemente. Además, ponernos de acuerdo en las decisiones importantes, como definir bien los roles y qué podía hacer cada usuario nos llevó tiempo.

Aparte de la organización, nos encontramos con **dos problemas** principalmente:

- **Querer abarcar demasiado (Scope Creep):** Al principio nos emocionamos queriendo meter demasiadas funciones y hacer módulos muy complejos. Tuvimos que frenar y priorizar lo realmente importante para no quedarnos sin tiempo. Es decir diseño y documentación
- **Unir dos mundos diferentes:** Fue difícil planificar cómo combinar las reglas fijas y estrictas de Laravel (como la seguridad de la base de datos) con un asistente de IA, cuyas respuestas pueden ser más impredecibles. Nos obliga a revisar el contenido constantemente.

Como **aprendizaje general**, este proceso nos ha servido para entender todo lo que implica planificar un proyecto real en Laravel trabajando en grupo. A nivel de equipo, hemos mejorado mucho en comunicación, reparto de tareas y en saber escuchar las opiniones de los demás para llegar a acuerdos. En lo personal, nos ha cambiado la mentalidad: hemos aprendido que no se puede empezar

a programar a lo loco sin antes haber pensado y diseñado bien la estructura en papel. Al final, el proyecto ha cambiado mucho desde nuestra idea inicial; empezamos buscando una web sencilla y hemos terminado diseñando una plataforma mucho más organizada, segura y profesional.

17. Referencias y fuentes consultadas

- ***En este apartado incluimos las fuentes que hemos usado durante el proyecto***

N.º	Fuente / título	Organización	URL / referencia	Uso en el documento
1	*Laravel Documentation (The Request Lifecycle / Architecture Concepts)*	*Laravel LLC*	*laravel.com*	*Justificación de la separación de responsabilidades y el flujo de peticiones en la sección 12.1 y 12.2.*
2	*Conventional Commits 1.0.0 Specification*	*Conventional Commits*	*conventionalcommits.org*	*Definición de la política de mensajes para el historial de versiones en la sección 5.2.*
3	*Laravel Security & Configuration Best Practices*	*Laravel LLC*	*laravel.com*	*Sustento técnico para la exclusión del archivo `.env` y el uso correcto de variables de entorno en la sección 5.3.*
4	*OpenAI API Guidelines: Safety & Prompt Injection Mitigation*	*OpenAI*	*openai.com*	*Análisis de riesgos, alucinaciones y límites operativos del asistente virtual en las secciones 13.3 y 13.4.*

18. Registro de decisiones del proyecto

- ***Aquí tenemos el registro de toma de decisiones y acciones del proyecto:***

ID	Fecha	Decisión	Alternativas	Justificación	Responsable / equipo
DEC-01	04 / 10 / 2026	**Asignación de roles internos** del equipo y reparto de las fases de investigación iniciales.	Trabajar de forma asíncrona sin un reparto de responsabilidades específico.	Optimiza los tiempos de entrega al permitir que cada integrante se especializa en investigar el alcance y necesidades de sus módulos asignados.	Equipo Completo
DEC-02	05 / 10 / 2026	**Investigación de requerimientos** por rol y definición del alcance de los 12 módulos.	Definir las funciones sobre la marcha según se fuera programando.	Evita el descontrol del alcance (*Scope Creep*) y asegura que entendemos perfectamente las necesidades lógicas de cada tipo de usuario desde el inicio.	Equipo Completo
DEC-03	07 / 10 / 2026	Diseño y comienzo del **modelo relacional de la Base de Datos** y bocetos del Frontend.	Crear una base de datos plana o diseñar interfaces sin tener clara la estructura de los datos.	Permite estructurar de forma limpia las relaciones de las tablas y garantiza que la interfaz visual responda exactamente a la lógica de la aplicación.	Equipo de desarrollo / Base de Datos
DEC-04	08 / 10 / 2026	Adopción del **Service Pattern en el Backend** y resolución del diagrama lógico de clases.	Utilizar el patrón MVC estándar concentrando el código en controladores tradicionales (*Fat Controllers*).	Evita la saturación de los controladores de Laravel, garantizando que el código de negocio complejo se gestione en clases independientes y	Equipo de desarrollo / Backend

				limpias.	
DEC-05	10 / 10 / 2026	Limitación del **asistente de IA a un rol consultivo** y delimitación de riesgos de integración.	Otorgar autonomía a la IA para ejecutar presupuestos o compras automáticas en el sistema.	Protege la plataforma contra posibles alucinaciones del modelo y blinda la trazabilidad con un registro de auditoría inmutable (Append-Only).	Equipo de desarrollo / IA
DEC-06	11 / 10 / 2026	Definición de las **políticas de Git, commits y seguridad** para el repositorio.	Permitir subidas libres a cualquier rama sin revisión de código previa.	Protege la integridad del repositorio principal mediante *Pull Requests* revisados y evita la filtración accidental de claves en archivos .env.	Equipo Completo / Git
DEC-07	12 / 10 / 2026	**Actualización final de la documentación**, cohesión de datos de última hora y cierre de la UT1.	Enviar el proyecto con la documentación inicial de la fase de planificación sin pulir el tono ni el orden.	Asegura que el proyecto cumpla con los estándares académicos requeridos, dejando la arquitectura lista para picar código directamente en la UT2.	Equipo Completo